

Verifiable and Redactable Blockchains With Fully Editing Operations

Jun Shen^{ID}, Xiaofeng Chen^{ID}, *Senior Member, IEEE*, Zheli Liu^{ID}, and Willy Susilo^{ID}, *Fellow, IEEE*

Abstract—Blockchain technology has been highly praised for its immutability, attracting considerable attention in the academic and industrial community. However, since permanent storage in blockchains would result in copyright disputes and harmful information spreads, it is desired to equip blockchains with mutability nowadays for legal and moral duty restrictions. A redactable blockchain is a variation, which enables the editing of block objects without affecting other blocks in the blockchain. Most existing redactable blockchains can only support editing operations such as modification and deletion, rather than insertion. However, insertion is necessary for some scenarios, especially when patching codes are needed for smart contracts and mistaken deletions occur. In addition, we argue that the verifiability of blockchains is also significantly important, especially for the redactable ones, in which one block may have multiple versions. Hence, an efficient mechanism is required to invalidate history versions and incent distributed adoptions of editing operations. In this paper, we propose a verifiable and redactable blockchain with fully editing operations for the first time. One distinguishable property is the simultaneous achievement of fully editability of block objects and verifiability of blockchain state, with acceptable extra cost. Specifically, we construct the redactable blockchain based on a double trapdoor chameleon hash family, enabling computationally efficient and key-exposure resistant block editing. Additionally, we combine trapdoorless universal accumulators and the largest sequence number principle to make the blockchain state verifiable. Furthermore, we present a comprehensive analysis and extensive experiments to demonstrate the security and feasibility of the proposed redactable blockchain.

Index Terms—Redactable blockchains, verifiability, fully editing, chameleon hash, cryptographic accumulator.

I. INTRODUCTION

BLOCKCHAIN is an emerging technology integration of distributed consensus, smart contract and cryptography, with excellent characteristics of decentralization, distribution,

transparency, etc. [1], [2], [3]. Such a technology has attracted extensive attention in recent years, turning up a wave of research and application in industry and academic societies, involving scopes from financial systems to healthcare or even politics [4], [5], [6].

A blockchain in its most primitive form is a decentralized and append-only data structure, recording immutable timestamped data objects. It is widely accepted that immutability is one of the most highly praised features of the blockchain, which underpins the security and transparency of blockchain applications, eliminating the need for third-party intermediaries and trust among entities in untrusted environments. With such a feature, blockchains would exercise persistent and substantial impact on economic and social systems [7], [8].

Despite the benefits, it is argued that an immutable permanent storage is not absolutely appropriate for all applications envisaged for the blockchain. In other words, the immutability might result in some unfavorable aspects that impede the wide adoption of the blockchain [9], [10]. Take Bitcoin as an instance, since its ability to attach arbitrary messages in transactions has been abused by malicious users, some improper or even illegal contents, such as privacy violations and child pornography, are recorded permanently. Scared of prosecutions due to holding these illegal contents, a considerable number of users refuse to download the blockchain, severely disrupting the Bitcoin ecosystem. Thus, redactions are necessary when old data objects become a liability. Furthermore, it is clear that our society disapproves of and is not ready for permanent storage yet. For example, the General Data Protection Regulation (GDPR) of the European Union imposes the “Right to Be Forgotten” as a key data subject right, which allows individuals the right to require an organization to delete their personal data without undue delay. As a consequence, it is no longer legal to use immutable blockchains to record personal data. Therefore, in light of various data protection policies and regulations, it is important to break the immutability of blockchains.

Ateniese et al. [11] first introduced the approach of re-writing and compressing the contents of a blockchain. It is universally learned that the immutability of blockchains results from the collision resistance property of the cryptographic hash function, which is employed to chain blocks by recursively nesting hash digests. Inspired, the main idea in [11] is to substitute a chameleon hash function [12] for the common ones. Thus, the block content can be adapted with the knowledge of trapdoor key, keeping the state of other blocks consistent. Subsequent to this breakthrough work,

Manuscript received 10 March 2022; revised 20 September 2022 and 4 April 2023; accepted 15 May 2023. Date of publication 21 June 2023; date of current version 29 June 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 61960206014 and Grant 62032012, in part by the Fundamental Research Funds for the Central Universities under Grant ZDRC2204, and in part by the Key Research and Development Program of Shaanxi under Grant 2020ZDLGY08-03. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Pete Burnap. (Corresponding author: Xiaofeng Chen.)

Jun Shen and Xiaofeng Chen are with the State Key Laboratory of Integrated Service Networks (ISN), Xidian University, Xi'an 710071, China (e-mail: demon_sj@126.com; xfchen@xidian.edu.cn).

Zheli Liu is with the College of Cyber Science and the College of Computer Science, Nankai University, Tianjin 300071, China (e-mail: liuzheli@nankai.edu.cn).

Willy Susilo is with the School of Computing and Information Technology, Institute of Cybersecurity and Cryptology, University of Wollongong, Wollongong, NSW 2522, Australia (e-mail: wsusilo@uow.edu.au).

Digital Object Identifier 10.1109/TIFS.2023.3288429

an increasing number of researchers have devoted great efforts to studying redactable blockchains involving various aspects, including but not limited to fine-grained control, instance redaction and permissionless setting [13], [14], [15], [16].

Most of these existing works support two types of editing operations, namely modification and deletion, while few of them have considered block insertion, which differs from conventional append operation and is essential to some scenarios. Take Bitcoin 2.0 applications for example, in which smart contracts and overlay applications would not work well for lack of scalability if the blockchain does not support insertion. Specifically, smart contracts are sequences of computer programs running on the blockchain database, programmed to self-execute when the conditions written in the source code are met. The DAO is such a contract, which was hacked and lost 3,641,694 ETH due to code vulnerabilities [17]. If a patching code could be inserted, the losses would be cut in time then. Besides, since redactable blockchains enable the delete operation, inevitably there might be some mistaken deletions. At this juncture, an insert operation is seriously needed. To this end, Dousti and Küpçü [18] attempted to propose a signature-based redactable blockchain supporting all the three editing operations. Unfortunately, such an approach is incompatible with block deletion and suffers from inconsistent blockchain state due to the imperfect verification strategies.

To the best of our knowledge, though extensive researches were conducted on redactable blockchains, it seems that there is no such research work on the verifiability of redactable blockchains. However, it is significantly required in the real-world applications. Conventionally, blockchain validating is executed mainly by checking the connectivity of hash values, i.e., the “chain”, without caring for the version of block objects. A blockchain, especially the chameleon hash based redactable one, can pass verification, even though it did not update modified blocks to the latest versions. Such a versioning problem is not involved in deletion and insertion, yet it is hard to ensure that these two operations are adopted distributively in time. This is because the blockchain adopting block deletion/insertion or not is always interconnected, unaffected the validation result or even subsequent block appending. From the above, we can see that without verifiable blockchain state, there would be no effective and strongly executive redactable blockchains. Therefore, it is of significant importance to accomplish verifiability of blockchain state for redactable blockchains supporting fully editing operations.

A. Our Contribution

In this paper, we aim to design a verifiable and redactable blockchain with fully editing operations, achieving full redactability and verifiability simultaneously. The contributions are summarized as follows.

- We propose a novel framework for redactable blockchain with verifiability and fully editability for the first time. In this framework, block objects support editing operations involving appending, inserting, modifying and deleting. Furthermore, editing operations are validated to

incent in-time adoption of redactions and blockchain state is verified to invalidate history versions of blocks.

- We employ a double trapdoor chameleon hash family to construct a verifiable and redactable blockchain with fully editing operations, which achieves history-independent and key-exposure resistant redactions. Extensive experiments show that the overhead imposed by such a blockchain is small and seems acceptable, and thus the feasibility of our approach.

B. Related Work

The concept of redactable blockchains was first put forward by Ateniese et al. [11], satisfying the demand of removing improper content and guaranteeing the right to be forgotten. In this work, they realized the re-writing of block content by finding collisions of a key-exposure resistant chameleon hash function, without affecting other blocks in the blockchain. However, Derler et al. [13] argued that such a block-level approach is too coarse-grained, and presented a fine-grained transaction-level editing solution of blockchains. This solution is based on the newly proposed policy-based chameleon hashes (PCH), which utilizes ciphertext-policy attribute-based encryption (CP-ABE) and chameleon-hashes with ephemeral trapdoors (CHETs) to strictly specify entities who can find hash collisions and thus to execute editing. However, such a solution suffers from unsatisfactory efficiency due to its several complex and costly techniques. An alternative to realize transaction-level editing is mutable transaction, which was introduced by Puddu et al. [14]. Such a transaction contains multiple inactive transaction versions and one activate version, where which one is active is controlled by mutability policy and enforced by consensus. When there is a request for redacting a transaction, miners check the policy and cooperate via a multi-party computation (MPC) protocol to decrypt and activate the required version. However, such an approach suffers from knavish mutability policies established by malicious users and faces the scalability issue caused by MPC [15]. In order to refrain from utilizing large-scale MPC, Deuber et al. [16] utilizes consensus-based voting methodology to propose the first redactable blockchain for permissionless setting. When an editing operation is proposed, miners vote in the blockchain via consensus. If enough votes are collected within the period limit, the redaction is approved to be applied. Such an approach is efficient but suffers from vote erasure attack [19], where redactions of voting blocks might erase the votes already collected for a redaction, putting the blockchain in an inconsistent state. Dousti and Küpçü [19] divided the existing solutions into moderated and unmoderated categories, and presented several attacks, including moderator circumvention attack against RSA-based construct [20], reversion attack on the chameleon hash based [11], [13], miner corruption attack against voting-based approach [16], etc.

Recently, Dousti and Küpçü [18] pointed out that all the existing works could only support two types of editing operations, i.e., modification and deletion. They attempt to give a signature-based moderated redactable blockchain that supports fully editing operations, including modification, deletion

and insertion. However, the proposed construct actually fails to be compatible with the delete operation. Besides, it suffers from inconsistent blockchain states due to defective verification strategies.

Other approaches [21], [22], [23] and applications [24], [25] concerning blockchain re-writings have also been researched recently. To the best of our knowledge, unfortunately, there is no appropriate redactable blockchain focusing on verifiable blockchain state and supporting all dynamic operations. In contrast, we explore redactable blockchains with verifiability and fully editing operations simultaneously.

C. Organization

The remaining sections are organized as follows. Section II reviewed some preliminaries, including chameleon hash functions, non-interactive succinct proofs and trapdoorless universal accumulators. In Section III, we describe the problem statement, involving the blockchain structure, the formal definition and security definitions of the proposed verifiable and redactable blockchain. The detailed description of the proposed blockchain is presented in section IV. In Section V, we analyze the security of the blockchain. Section VI discusses the evaluation of performance. Finally, the conclusion is drawn in Section VII.

II. PRELIMINARIES

We review preliminaries including chameleon hash functions, non-interactive succinct proofs and trapdoorless universal accumulators.

A. Chameleon Hash Functions

Chameleon hash functions are put forward by Krawczyk and Rabin [26], which are trapdoor one-way hash functions and allow the trapdoor holder to figure out collisions for arbitrary given input. However, such a function suffers from the key exposure problem. Plenty of researchers have devoted considerable attention to addressing the problem and improving the security of chameleon hash functions [12], [27]. Chen et al. also proposed a computationally efficient and key-exposure resistant double trapdoor chameleon hash family [28], which is a tuple of following algorithms $\mathcal{CH} = (\text{KGen}, \text{HGen}, \text{RHGen}, \text{Verify}, \text{Adapt})$.

- $\mathcal{CH}.\text{KGen}(1^\lambda) \rightarrow (tk, hk)$. On input the security parameter λ , it outputs trapdoor key $tk = (x, t)$ and hash key $hk = Y$, where $x, t \xleftarrow{\$} \mathbb{Z}_q$, $Y = xP$, P is generator of group $\mathbb{G}_0$.
- $\mathcal{CH}.\text{HGen}(tk, m) \rightarrow (h, \xi)$. On input trapdoor tk and message m , it computes $h = tP$ as the hash value. Then pick $k \xleftarrow{\$} \mathbb{Z}_q$, compute checking string $\xi = (r, K)$, where $K = kP$, $r = t - H_{\mathbb{Z}_q}(m, K) \cdot (k + x)$ and $H_{\mathbb{Z}_q} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$.
- $\mathcal{CH}.\text{RHGen}(m, \xi, hk) \rightarrow (h)$. In this algorithm, h can be calculated without tk via $h = H_{\mathbb{Z}_q}(m, K) \cdot (K + hk) + rP$.

- $\mathcal{CH}.\text{Verify}(h, m, \xi, hk) \rightarrow (\{0, 1\})$. With the inputs, it checks $h \stackrel{?}{=} H_{\mathbb{Z}_q}(m, K) \cdot (K + hk) + rP$. Output 1 if the equation holds, and 0 otherwise.
- $\mathcal{CH}.\text{Adapt}(tk, h, m, \xi, m') \rightarrow (\xi')$. This algorithm finds a collision $\xi' = (r', K')$ for a new message m' , where $r' = t - H_{\mathbb{Z}_q}(m', K') \cdot (k' + x)$, $K' = k'P$ and $k' \xleftarrow{\$} \mathbb{Z}_q$.

Note that, in such a chameleon hash function, t can be regarded as the specific trapdoor for hash value $h = tP$. With the same (x, Y) pair, different t 's correspond to different hash values, which relieves the storage burden of trapdoor holder.

The collision resistance of this double trapdoor chameleon hash family allows the adversary to access the collision-finding oracle to see arbitrary collisions also for the target hash, but not for the target message. Then the adversary cannot find any collisions for the messages which have not been queried to the oracle.

Definition 1 (Collision Resistance): The collision resistance of the double trapdoor chameleon hash family $\mathcal{CH}$ is based on the following experiment:

```

Exp $\mathcal{CH}, \mathcal{A}$ CollRes( $1^\lambda$ )
   $(tk, hk) \leftarrow \mathcal{CH}.\text{KGen}(1^\lambda)$ 
   $\mathcal{Q} \leftarrow \emptyset$ 
   $(m^*, \xi^*, m'^*, \xi'^*, h^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Adapt}(tk, \cdot, \cdot, \cdot)}}(hk)$ 
  where  $\mathcal{O}_{\text{Adapt}(tk, h, m, \xi, m')}$ 
    return  $\perp$ , if  $\mathcal{CH}.\text{Verify}(h, m, \xi, hk) \neq 1$ 
     $\xi' \leftarrow \mathcal{CH}.\text{Adapt}(tk, h, m, \xi, m')$ 
    return  $\perp$ , if  $\xi' = \perp$ 
     $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{m, m'\}$ 
    return  $\xi'$ 
  return 1, if  $\mathcal{CH}.\text{Verify}(h^*, m^*, \xi^*, hk) = 1 \wedge$ 
     $\mathcal{CH}.\text{Verify}(h^*, m'^*, \xi'^*, hk) = 1 \wedge$ 
     $m^* \neq m'^* \wedge m^* \notin \mathcal{Q}$ 
  return 0

```

$\mathcal{CH}$ is said to be collision resistant if for any PPT adversary $\mathcal{A}$ and security parameter λ , it holds that

$$\Pr[\mathbf{Exp}_{\mathcal{CH}, \mathcal{A}}^{\text{CollRes}}(1^\lambda) = 1] \leq \text{negl}(\lambda),$$

where $\text{negl}(\cdot)$ is a negligible function.

The key-exposure freeness of the double trapdoor chameleon hash family allows the adversary to access the collision-finding oracle, and the adversary cannot figure out the trapdoor key.

Definition 2 (Key-exposure Freeness): The key-exposure freeness of the double trapdoor chameleon hash family $\mathcal{CH}$ is based on the following experiment:

```

Exp $\mathcal{CH}, \mathcal{A}$ KExpF( $1^\lambda$ )
   $(tk, hk) \leftarrow \mathcal{CH}.\text{KGen}(1^\lambda)$ 
   $(tk^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Adapt}(tk, \cdot, \cdot, \cdot)}}(hk)$ 
  where  $\mathcal{O}_{\text{Adapt}(tk, h, m, \xi, m')}$ 
    return  $\perp$ , if  $\mathcal{CH}.\text{Verify}(h, m, \xi, hk) \neq 1$ 
     $\xi' \leftarrow \mathcal{CH}.\text{Adapt}(tk, h, m, \xi, m')$ 
    return  $\perp$ , if  $\xi' = \perp$ 
    return  $\xi'$ 
  return 1, if  $tk^* = tk$ 
  return 0

```

$\mathcal{CH}$ is said to be key-exposure free if for any PPT adversary $\mathcal{A}$ and security parameter λ , it holds that

$$\Pr \left[\text{Exp}_{\mathcal{CH}, \mathcal{A}}^{\text{KEPF}}(1^\lambda) = 1 \right] \leq \text{negl}(\lambda).$$

B. Non-Interactive Succinct Proofs

The cryptographic accumulator employed in our verifiable and redactable blockchain is based on two non-interactive, succinct and efficiently verifiable proofs for hidden order groups, namely NI-PoE and NI-PoKE [29].

NI-PoE is an argument for a discrete-log relation $\mathcal{R}_{\text{PoE}} = \{x \in \mathbb{Z}, u, w \in \mathbb{G}_1 : u^x = w\}$, described as follows, where $H_{\text{prime}} : \{0, 1\}^* \rightarrow \text{Primes}(\lambda)$ is a hash function and λ is the security parameter.

- **NI-PoE.Prove** $(x, u, w) \rightarrow (Q)$. With the inputs, it computes $l \leftarrow H_{\text{prime}}(x, u, w)$ and $q \leftarrow \lfloor x/l \rfloor$, and outputs $Q \leftarrow u^q$ as the proof.
- **NI-PoE.Verify** $(x, u, w, Q) \rightarrow (\{0, 1\})$. The algorithm computes $l \leftarrow H_{\text{prime}}(x, u, w)$ and $r \leftarrow x \bmod l$. Check $Q^l u^r \stackrel{?}{=} w$ and output 1 if the equation holds.

NI-PoE can be adapted to construct NI-PoKE, which is an argument of knowledge for $\mathcal{R}_{\text{PoKE}} = \{(u, w \in \mathbb{G}_1) : x \in \mathbb{Z} : u^x = w\}$. $H_{\mathbb{G}_1} : \{0, 1\}^* \rightarrow \mathbb{G}_1$ and $H_\lambda : \{0, 1\}^* \rightarrow [0, 2^\lambda)$ are two hash functions. The argument of knowledge is presented as follows.

- **NI-PoKE.Prove** $(x, u, w) \rightarrow (z, r, Q)$. With the inputs, it computes $u' \leftarrow H_{\mathbb{G}_1}(u, w)$, $z \leftarrow u'^x$, $l \leftarrow H_{\text{prime}}(u, w, z)$, $a \leftarrow H_\lambda(u, w, z, l)$ and $q \leftarrow \lfloor x/l \rfloor$, and outputs $(z, Q \leftarrow (uu'^a)^q, r \leftarrow x \bmod l)$ as the proof.
- **NI-PoKE.Verify** $(u, w, z, r, Q) \rightarrow (\{0, 1\})$. The algorithm computes $u' \leftarrow H_{\mathbb{G}_1}(u, w)$, $l \leftarrow H_{\text{prime}}(u, w, z)$ and $a \leftarrow H_\lambda(u, w, z, l)$. Output 1 if $Q^l (uu'^a)^r = wz^a$.

C. Trapdoorless Universal Accumulators

The trapdoorless universal accumulator [29] extends the classic RSA-based accumulator [30], [31], supports both membership and non-membership proofs and is designed for a decentralized setting. Since our blockchain does not involve all algorithms of such an accumulator, here we just present a simplified version, i.e., $\mathcal{UA} = \langle \text{Setup}, \text{Add}, \text{Del}, \text{MWit}, \text{N-MWit}, \text{MVfy}, \text{N-MVfy} \rangle$ built on a group $\mathbb{G}_1$ of unknown order.

- **UA.Setup** $(1^\lambda) \rightarrow (S, A(S))$. With security parameter λ , the setup algorithm outputs the element set $S \leftarrow \emptyset$ and initializes accumulator state $A(S) \leftarrow h$, where $h \xleftarrow{\$} \mathbb{G}_1$.
- **UA.Add** $(A(S), S, x) \rightarrow (A(S'))$. This algorithm is for element addition. If $x \in S$, output $A(S') \leftarrow A(S)$ directly. Otherwise, update $A(S') \leftarrow A(S)^x$.
- **UA.Del** $(A(S), S, x) \rightarrow (A(S'))$. The element deletion algorithm outputs $A(S') \leftarrow A(S)$ if $x \notin S$. Otherwise, let $S' \leftarrow S \setminus \{x\}$, compute $x^* \leftarrow \prod_{x_i \in S'} x_i$, and update accumulator $A(S') \leftarrow h^{x^*}$.
- **UA.MWit** $(A(S), S, x) \rightarrow (\pi)$. To prove the membership of x in S , the algorithm computes $x^* \leftarrow \prod_{x_i \in S \setminus \{x\}} x_i$

and $w \leftarrow h^{x^*}$. Output $\pi \leftarrow \text{NI-PoE.Prove}(x, w, A(S))$ as the membership proof.

- **UA.N-MWit** $(A(S), S, x) \rightarrow (\pi)$. To show that $x \notin S$, it computes $x^* \leftarrow \prod_{x_i \in S} x_i$, utilizes the extended euclidean algorithm $\text{Exgcd}(x, x^*)$ to obtain $\alpha, \beta \in \mathbb{Z}$, and lets $d \leftarrow h^\alpha$, $v \leftarrow A(S)^\beta$. Output proof $\pi = (\pi_d, \pi_h) \leftarrow (\text{NI-PoKE.Prove}(A(S), v, \beta), \text{NI-PoE}(x, d, hv^{-1}))$.
- **UA.MVfy** $(A(S), x, w, \pi) \rightarrow (\{0, 1\})$. The algorithm takes membership proofs as inputs and outputs 1 if $\text{NI-PoE.Verify}(x, w, A(S), \pi) \rightarrow 1$.
- **UA.N-MVfy** $(A(S), x, v, d, \pi) \rightarrow (\{0, 1\})$. On input non-membership proofs, the algorithm outputs 1 if both $\text{NI-PoKE.Verify}(A(S), v, \pi_d) \rightarrow 1$ and $\text{NI-PoE.Verify}(x, d, hv^{-1}, \pi_h) \rightarrow 1$ hold.

Note that in the proposed redactable blockchain, it is unnecessary to maintain the element set S of the accumulator, since these elements can be parsed from the blocks. Besides, since the deletion algorithm is costly without the knowledge of group order, it is executed by the regulator with the RSA group order in the proposed blockchain.

III. PROBLEM STATEMENT

We first introduce the proposed blockchain structure. Then, we formalize the syntax and security definitions of the proposed blockchain.

A. The Proposed Blockchain Structure

Similar to notations in [11] and [32], we define a block as a tuple $B = \langle p, m, i, A, w, ctr, \xi \rangle$, shown in Fig. 1. $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a cryptographic collision-resistant hash function and $\mathcal{CH}$ is a chameleon one. Then a block is valid if

$$\text{validblock}^D(B) := H_1(ctr, \mathcal{CH}.\text{RHGen}(cont, \xi, hk)) < D.$$

Here, p is hash of the front block, $cont = p||m||i||A||w$, m indicates the Merkle root of transactions included in B , i is a globally unique sequence number, A denotes the accumulator state of the whole chain up to B and w is the witness for A , ctr is the nonce winning Proof of Work (PoW) game, ξ is the checking string corresponding to chameleon hash value generated on $cont$, and D is the difficulty level.

A blockchain $\mathcal{C}$ is a chain of blocks in essential, which is usually extended to a longer one by appending blocks to the rightmost position. The rightmost block is called the head of the chain. Suppose that $\mathcal{C}$ with $\text{Head}(\mathcal{C}) = B$ is extended to $\mathcal{C}' := \mathcal{C}||B'$, where $B' := \langle p', m', i', A', w', ctr', \xi' \rangle$ such that $p' = H_1(ctr, \mathcal{CH}.\text{RHGen}(cont, \xi, hk))$.

In addition, $\text{Len}(\mathcal{C})$ denotes the number of blocks contained in $\mathcal{C}$. Suppose that $\text{Len}(\mathcal{C}) = n$, then for $0 \leq k \leq n$, ${}^k\mathcal{C}$ is the chain resulted from cutting off the leftmost k blocks from $\mathcal{C}$, while $\mathcal{C}^{\lceil k}$ indicates the chain derived from removing the rightmost k blocks. Thus, $\mathcal{C}' := \mathcal{C}^{[n-k+1]}||B^*||^{k-1}\mathcal{C}$ extends $\mathcal{C}$ by inserting a new block B^* into the k -th position. Besides, $\mathcal{C}' := \mathcal{C}^{[n-k+1]}||B^*||^k\mathcal{C}$ edits $\mathcal{C}$ by modifying the k -th block, and $\mathcal{C}' := \mathcal{C}^{[n-k+1]}||^k\mathcal{C}$ shrinks $\mathcal{C}$ by deleting the k -th block.

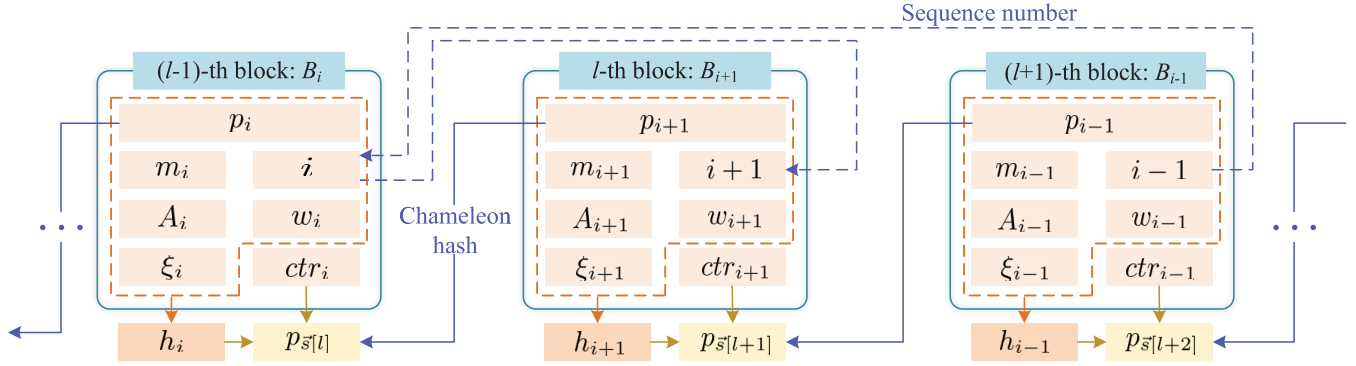


Fig. 1. The proposed verifiable and redactable blockchain structure.

B. The Formal Definition

Definition 3: The verifiable and redactable blockchain with fully editing operations consists of algorithms defined below:

- **Setup** (1^λ) $\rightarrow (pp)$. This probabilistic algorithm takes the security parameter λ as input and outputs system parameter pp .
- **KGenR** (pp) $\rightarrow (sk, pk)$. The input of this probabilistic algorithm is pp , and the output is secret and public key pair of regulator $\mathcal{R}$.
- **KGenB** (sk, pk, i) $\rightarrow (tk_i, hk)$. On input the key pair of $\mathcal{R}$ and a globally unique sequence number, this probabilistic algorithm outputs a specific chameleon hash key pair for the block with sequence number i .
- **Append** ($\mathcal{C}, \text{Len}(\mathcal{C}), m, ctr, (tk_i, hk)$) $\rightarrow (\mathcal{C}')$. This deterministic algorithm is executed to append a new block with Merkle root m and nonce ctr to $\mathcal{C}$, outputting the updated blockchain $\mathcal{C}'$.
- **ValApp** ($B_i, \mathcal{C}, \text{Len}(\mathcal{C})$) $\rightarrow (\{0, 1\})$. This is a probabilistic algorithm, verifying the validity of B_i appended to $\mathcal{C}$.
- **Insert** ($\mathcal{C}, \text{Len}(\mathcal{C}), m, ctr, l, (tk_i, tk_{s[l]}, hk)$) $\rightarrow (\mathcal{C}')$. This algorithm is deterministic. It inserts a new block with m and ctr in blockchain $\mathcal{C}$ to obtain $\mathcal{C}'$.
- **Vallns** ($B_i, B'_{s[l]}, l, \mathcal{C}, \text{Len}(\mathcal{C})$) $\rightarrow (\{0, 1\})$. This probabilistic algorithm verifies the validity of insert operations.
- **Modify** ($\mathcal{C}, \text{Len}(\mathcal{C}), m', l, (tk_{s[l]}, hk), \phi(N)$) $\rightarrow (\mathcal{C}')$. This deterministic algorithm is to modify the l -th block in $\mathcal{C}$.
- **ValMod** ($B_i, l, \mathcal{C}, \text{Len}(\mathcal{C})$) $\rightarrow (\{0, 1\})$. This probabilistic algorithm verifies the validity of modifications.
- **Delete** ($\mathcal{C}, \text{Len}(\mathcal{C}), L, m, ctr, (tk_{s_a}, tk_i, hk), \phi(N)$) $\rightarrow (\mathcal{C}')$. This deterministic algorithm is used to delete blocks indicated by set L in $\mathcal{C}$.
- **ValDel** ($B_i, L, \mathcal{C}, \text{Len}(\mathcal{C})$) $\rightarrow (\{0, 1\})$. This probabilistic algorithm verifies the validity of deletions.
- **ValChain** ($\mathcal{C}, \text{Len}(\mathcal{C})$) $\rightarrow (\{0, 1\})$. This probabilistic algorithm verifies the validity of blockchain $\mathcal{C}$.

C. Security Definitions

A secure verifiable and redactable blockchain with fully editing operations should satisfy properties in two aspects. One is block security indicating the unforgeability of individual block, and the other is blockchain security including chain sequence growth, chain sequence quality and editable common presequence. The definitions are as follows.

Definition 4 (Block Security): Blocks in a verifiable and redactable blockchain with fully editing operations are secure if it is infeasible for any PPT adversary $\mathcal{A}$ to put a forged block B^* concerning $cont_i^* = p_i || m^* || i || A^* || w^*$ in blockchain $\mathcal{C}$ approved by honest users, while $cont_i = p_i || m_i || i || A_i || w_i$ is the actual content. That is, for any security parameter λ and all editing operations, there is

- If $\mathcal{A}$ attempts to append B^* in $\mathcal{C}$, then

$$\Pr \left[\mathcal{A}^{\mathcal{O}(tk_i, cont_i)}(hk, \mathcal{C}, \text{Len}(\mathcal{C}), cont^*, ctr) \rightarrow \mathcal{C}^* \right. \\ \wedge \text{ValApp}(B^*, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow 1 \\ : (tk_i, hk) \leftarrow \text{KGenB}(sk, pk, i), (sk, pk) \leftarrow \text{KGenR}(pp), pp \leftarrow \text{Setup}(1^\lambda) \Big] \leq \text{negl}(\lambda),$$

where $\mathcal{C}^* := \mathcal{C} || B^*$.

- If $\mathcal{A}$ attempts to insert B^* at the l -th position in $\mathcal{C}$, then

$$\Pr \left[\mathcal{A}^{\mathcal{O}(tk_i, cont_i)}(hk, \mathcal{C}, \text{Len}(\mathcal{C}), cont^*, ctr, l) \rightarrow \mathcal{C}^* \right. \\ \wedge \text{Vallns}(B^*, B'_{s[l]}, l, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow 1 \\ : (tk_i, hk) \leftarrow \text{KGenB}(sk, pk, i), (sk, pk) \leftarrow \text{KGenR}(pp), pp \leftarrow \text{Setup}(1^\lambda) \Big] \leq \text{negl}(\lambda),$$

where $\mathcal{C}^* := \mathcal{C}^{[n-l+1]} || B^* || B'_{s[l]} || \mathcal{C}$.

- If B^* pretend to be the modification target at the l -th position in $\mathcal{C}$, then

$$\Pr \left[\mathcal{A}^{\mathcal{O}(tk_{s[l]}, \cdot)}(hk, \mathcal{C}, \text{Len}(\mathcal{C}), cont^*, l) \rightarrow \mathcal{C}^* \right. \\ \wedge \text{ValMod}(B^*, l, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow 1 \\ : (tk_{s[l]}, hk) \leftarrow \text{KGenB}(sk, pk, s[l]), (sk, pk) \leftarrow \text{KGenR}(pp), pp \leftarrow \text{Setup}(1^\lambda) \Big] \leq \text{negl}(\lambda),$$

where “ $\cdot$ ” indicates $cont_{s[l]}$ and $cont'_{s[l]}$, and $\mathcal{C}^* := \mathcal{C}^{[n-l+1]} || B^* || \mathcal{C}$.

- If $\mathcal{A}$ attempts to output B^* as the block recording deletion event in $\mathcal{C}$, then

$$\Pr \left[\mathcal{A}^{\mathcal{O}(tk_i, cont_i)}(hk, \mathcal{C}, \text{Len}(\mathcal{C}), L, cont^*, ctr) \rightarrow \mathcal{C}^* \right. \\ \wedge \text{ValDel}(B^*, L, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow 1 \\ : (tk_i, hk) \leftarrow \text{KGenB}(sk, pk, i), (sk, pk) \leftarrow \text{KGenR}(pp), pp \leftarrow \text{Setup}(1^\lambda) \Big] \leq \text{negl}(\lambda),$$

where $\mathcal{C}^* := \mathcal{C}^{[n-L_{min}+1]} || B'_{s_a} || \mathcal{C}^{[L_{max}+1]} || B^*$.

Here, **Setup** is honest, $\mathcal{O}$ is the chameleon hash oracle, tk_i is specific trapdoor for B_i and $\text{negl}(\cdot)$ is a negligible function.

Inspired by [16] and [32], we define blockchain security as follows. First, since blocks can be deleted and the state consensus in the proposed blockchain is the largest sequence number principle instead of the longest chain rule, we introduce a new definition that is suitable for the current scenario.

Definition 5 (Chain Sequence Growth): Given local blockchains $\mathcal{C}_1$ and $\mathcal{C}_2$ possessed by two honest users at the onset of two rounds ro_1 and ro_2 respectively, where ro_2 is at least x rounds ahead of ro_1 . Then, it holds that

$$\max_{j \in [\text{Len}(\mathcal{C}_2)]} \tilde{s}[j] - \max_{j \in [\text{Len}(\mathcal{C}_1)]} \tilde{s}[j] \geq x \cdot \tau,$$

where $\tau \in (0, 1]$ is the speed coefficient.

Next, the chain quality property in [32] states that the ration of adversarial blocks in arbitrary segment of blockchains held by honest users is strictly controlled. Similarly, we define Chain Sequence Quality as follows.

Definition 6 (Chain Sequence Quality): Given a number of y blocks with successive sequence numbers in blockchain possessed by an honest user, the ratio of adversarial blocks in this bunch is at most γ , where $\gamma \in (0, 1]$ is the quality coefficient and indicates the fraction of network resources controlled by the adversary.

The final is editable common presequence adapted from the common prefix property [32] and editable common prefix [16], which is defined to account for edits in the blockchain.

Definition 7 (Editable Common Presequence): Given local blockchains $\mathcal{C}_1$ and $\mathcal{C}_2$ possessed by two honest users at the onset of rounds ro_1 and ro_2 , where $ro_1 \leq ro_2$. Then for the editable common presequence parameter $\kappa \in \mathbb{N}$, one of the following is satisfied:

- Blocks with sequence number no larger than $\max_{j \in [\text{Len}(\mathcal{C}_1)]} \tilde{s}[j] - \kappa$ in $\mathcal{C}_1$ compose the presequence of $\mathcal{C}_2$, which means that only append and insert operations are executed in rounds between ro_1 and ro_2 .
- For each block B^* contained in the segment of $\mathcal{C}_1$ with sequence number no larger than $\max_{j \in [\text{Len}(\mathcal{C}_1)]} \tilde{s}[j] - \kappa$ but not in $\mathcal{C}_2$, there must be an authorized modification or deletion executed in rounds between ro_1 and ro_2 .

IV. THE PROPOSED VERIFIABLE AND REDACTABLE BLOCKCHAIN

We start with the high-level description of the proposed blockchain, and then give the complete description.

A. High-Level Description

We argue that directly utilizing a chameleon hash function to chain blocks is not sufficient for constructing a secure redactable blockchain. Though such an approach re-writes blocks by finding hash collisions, unaffected other blocks in the blockchain, it causes problems of lazy redaction update and valid histories.

We employ the largest sequence number principle as a substitute for the longest chain rule to incent users to compete for profits based on the latest blockchain version, addressing the lazy redaction update problem. In addition, to invalidate history versions of redacted blocks, we equip the redactable blockchain with efficient verifiability via the trapdoorless universal accumulator, which is utilized as a commitment to all blocks in the blockchain.

Specifically, when there are append and insert operations, the miner uses $\mathcal{U.A.Add}$ to update accumulator state and $\mathcal{U.A.MWit}$ to give the membership witness of the block. Since insertion breaks the connectivity of “chain”, $\mathcal{CH.Adapt}$ should be executed for the block right behind the inserted one. Then, to modify a block, $\mathcal{U.A.Del}$ is run to erase the history version, and $\mathcal{U.A.Add}$ is to add the new one into the accumulator. After that, the miner runs $\mathcal{U.A.MWit}$ and $\mathcal{U.A.N-MWit}$ to give the non-membership and membership witnesses of invalid and valid versions, separately. As for deletion, merely utilizing $\mathcal{U.A.Del}$ to update the accumulator state makes intuitive sense. However, the lazy redaction update issue still exists since the latest block does not change before and after deletion. Thus, appending an extra block recording the deletion event and updating the largest sequence number are necessary. As supplementary, the miner executes $\mathcal{U.A.Add}$ for the recording block, and runs $\mathcal{U.A.N-MWit}$ and $\mathcal{U.A.MWit}$ to generate witnesses for the deleted and appended blocks.

B. The Complete Description

In order for the proposed verifiable and redactable blockchain with fully dynamic operations to work, we start with adjusting the consensus of wildly accepted blockchains like Bitcoin. On the one hand, we still utilize PoW as the membership selection algorithm to determine which miner’s primary goal is used to propose a new block. On the other hand, the state consensus is now the largest sequence number principle. Such a novel consensus states that users must select the blockchain with the largest sequence number as the canonical one and extend it subsequently.

For simplicity, we use $\mathcal{M}$ to signify blockchain miner, which is a user competing for profits through PoW. Besides, $\mathcal{V}$ represents users failing to solve the PoW puzzle in the current round, which are responsible for verifying the validity of the blockchain extension. In addition, $\mathcal{R}$ is a trusted network regulator, holding trapdoor key for executing editing operations in blockchains.

The concrete construct consists of the following algorithms.

- **Setup** (1^λ) $\rightarrow (pp)$. Given the security parameter λ , it runs as follows:
 - 1) Generate a group $\mathbb{G}_0$ of prime order q , and pick a generator $P \in \mathbb{G}_0$ randomly.
 - 2) Generate a strong RSA modulus $N = q_0 \cdot q_1$, where $q_0 = 2q'_0 + 1$ and $q_1 = 2q'_1 + 1$ are large primes only known to $\mathcal{R}$, forming the trapdoor $\phi(N) = 4q'_0q'_1$ of accumulator. Here the multiplicative group $\mathbb{Z}_N^*$ of modulus N is a hidden order group for all entities other than $\mathcal{R}$.

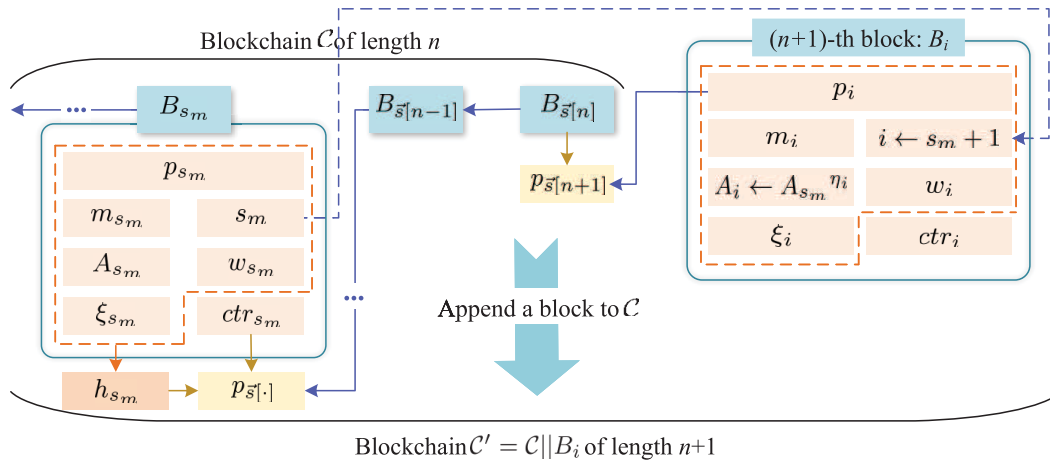


Fig. 2. The append operation.

- 3) Select a generator $h \in \mathbb{Z}_N^*$ randomly, and initialize the accumulator state as $A_0 \leftarrow h$.

- 4) Select two cryptographic hash functions:

$$\begin{cases} H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda \\ H_{\text{prime}} : \{0, 1\}^* \rightarrow \text{Primes}(\lambda) \end{cases}$$

- 5) Output the public parameter:

$$pp \leftarrow \{\mathbb{G}_0, q, P, \mathbb{Z}_N^*, h, H_1, H_{\text{prime}}, A_0\}.$$

- $\text{KGenR}(pp) \rightarrow (sk, pk)$. On input pp , the secret and public key pair for $\mathcal{R}$ is generated as follows:

- 1) $\mathcal{R}$ picks $x \xleftarrow{\$} \mathbb{Z}_q$ and calculates $Y = xP$.
- 2) The key pair is $(sk, pk) = (x, Y)$.

- $\text{KGenB}(sk, pk, i) \rightarrow (tk_i, hk)$. On input (sk, pk) of $\mathcal{R}$ and the globally unique sequence number i , it runs as follows:

- 1) $\mathcal{R}$ chooses $t_i \xleftarrow{R} \mathbb{Z}_q$ as a specialized trapdoor.
- 2) The specific chameleon hash key pair for block with sequence number i is $(tk_i, hk) = ((sk, t_i), pk)$.

- $\text{Append}(\mathcal{C}, \text{Len}(\mathcal{C}), m, ctr, (tk_i, hk)) \rightarrow (\mathcal{C}')$. Inputs of the algorithm are a blockchain $\mathcal{C}$ with $\text{Len}(\mathcal{C}) = n$, chameleon hash key pair (tk_i, hk) . To append B_i with Merkle root m and PoW winning nonce ctr , the algorithm executes as follows, which is depicted in Fig. 2 and described detailedly in Algorithm 1:

- 1) $\mathcal{M}$ calculates p_i through $\text{Head}(\mathcal{C}) = B_{s[n]}$, and obtains the sequence number by plus 1 for the largest one in $\mathcal{C}$.
- 2) $\mathcal{M}$ updates accumulator state to A_i by adding $\eta_i \leftarrow H_{\text{prime}}(m || i)$ derived from B_i , and gives the corresponding witness w_i .
- 3) $\mathcal{R}$ generates the checking string ξ_i corresponding to $cont_i$ in B_i via tk_i .
- 4) $\mathcal{M}$ outputs the updated blockchain $\mathcal{C}' \leftarrow \mathcal{C} || B_i$.

- $\text{ValApp}(B_i, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow (\{0, 1\})$. On input the appended block B_i and $\mathcal{V}$'s local chain $\mathcal{C}$ with $\text{Len}(\mathcal{C}) = n$, it runs as follows and is described in Algorithm 2.

- 1) $\mathcal{V}$ verifies the validity of sequence number, connectivity with the front block, correctness of the PoW

Algorithm 1 Append Block

Require: $\mathcal{C}, \text{Len}(\mathcal{C}), m, ctr, (tk_i, hk)$.

Ensure: $\mathcal{C}'$.

- 1: parse $\mathcal{C}$ as $(B_1, B_2, \dots, B_{s[n]})$;
 - 2: parse $B_{s[n]} := \langle p_{s[n]}, ctr_{s[n]}, m_{s[n]}, \bar{s}[n], \xi_{s[n]}, A_{s[n]}, w_{s[n]} \rangle$;
 - 3: $cont_{s[n]} \leftarrow p_{s[n]} || m_{s[n]} || \bar{s}[n] || A_{s[n]} || w_{s[n]}$;
 - 4: $h_{s[n]} \leftarrow \mathcal{CH}.\text{RHGen}(cont_{s[n]}, \xi_{s[n]}, hk)$;
 - 5: $p_i \leftarrow H_1(ctr_{s[n]} || h_{s[n]})$; {hash of the front block}
 - 6: $s_m \leftarrow \max_{j \in [n]} \bar{s}[j], i \leftarrow s_m + 1$; {sequence number}
 - 7: $\eta_i \leftarrow H_{\text{prime}}(m || i), A_i \leftarrow A_{s_m}^{\eta_i}$; {accumulator state}
 - 8: $Q_i \leftarrow \text{NI-PoE.Prove}(\eta_i, A_{s_m}, A_i)$;
 - 9: $w_i \leftarrow Q_i$; {witness of A_i }
 - 10: $(h_i, \xi_i) \leftarrow \mathcal{CH}.\text{HGen}(tk_i, p_i || m || i || A_i || w_i)$; {checking string}
 - 11: $B_i := \langle p_i, ctr, m, i, \xi_i, A_i, w_i \rangle$;
 - 12: $\mathcal{C}' \leftarrow \mathcal{C} || B_i$;
 - 13: **return** $\mathcal{C}'$
-

solution, and consistency of the accumulator state and witnesses.

- 2) Outputs decision 1 if all the above conditions are met.

- $\text{Insert}(\mathcal{C}, \text{Len}(\mathcal{C}), m, ctr, l, (tk_i, tk_{s[l]}, hk)) \rightarrow (\mathcal{C}')$. The inputs are a blockchain $\mathcal{C}$ with $\text{Len}(\mathcal{C}) = n$, chameleon hash key pairs (tk_i, hk) for block B_i and $(tk_{s[l]}, hk)$ for $B_{s[l]}$. To insert B_i with m and ctr at the l -th position in $\mathcal{C}$, it is schematically shown in Fig. 3 and executed as follows, of which the details are in Algorithm 3.

- 1) $\mathcal{M}$ sticks previous hash value of the original l -th block $B_{s[l]}$, and obtains sequence number i , accumulator state A_i and witness w_i by the similar way in **Append**.
- 2) $\mathcal{R}$ generates checking strings ξ_i for B_i via tk_i and adapts $\xi'_{s[l]}$ for $B'_{s[l]}$ via $tk_{s[l]}$, since front block of the original l -th one changes.
- 3) $\mathcal{M}$ updates $\mathcal{C}$ to $\mathcal{C}' \leftarrow \mathcal{C}^{[n-l+1]} || B_i || B'_{s[l]} || \mathcal{C}$.

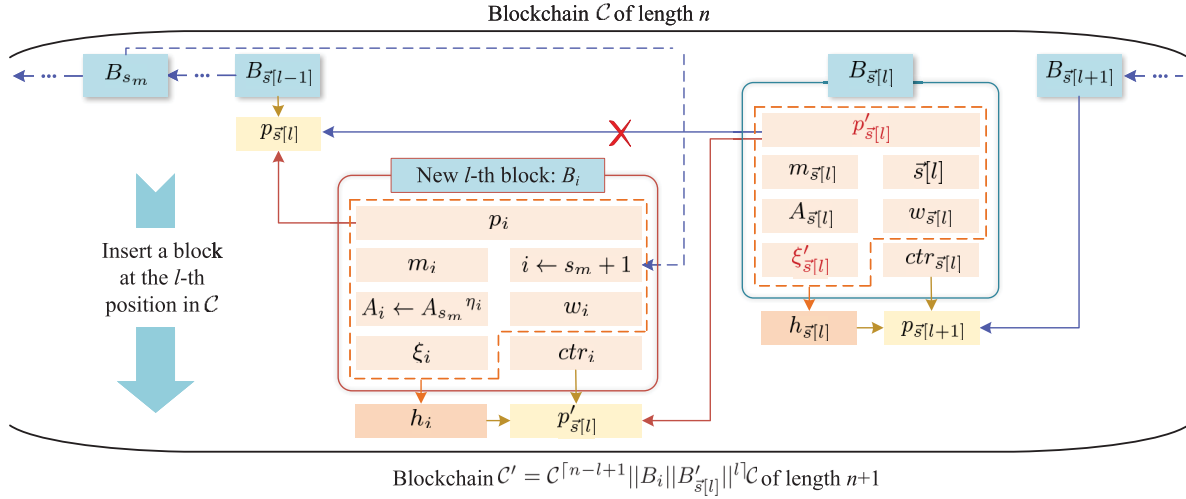


Fig. 3. The insert operation.

Algorithm 2 Validate Append Operation**Require:** $B_i, \mathcal{C}, \text{Len}(\mathcal{C})$.**Ensure:** $\{0, 1\}$.

```

1: parse  $B_i := \langle p_i, ctr_i, m_i, i, \xi_i, A_i, w_i \rangle, w_i := Q_i$ ;
2: if block structure is valid then
3:    $s_m \leftarrow \max_{j \in [n]} \vec{s}[j]$  of  $\mathcal{C}$ ;
4:   parse  $B_{s_m} := \langle \dots, A_{s_m}, \cdot \rangle$ ;
5:    $cont_i \leftarrow p_i || m_i || i || A_i || w_i$ ;
6:    $h_i \leftarrow \mathcal{CH}.\text{RHGen}(cont_i, \xi_i, hk)$ ;
7:   if  $i == s_m + 1$  and  $p_i == H_1(ctr_{s_m} || h_{s_m})$  and
      $H_1(ctr_i || h_i) < D$  then
8:      $\eta_i \leftarrow H_{\text{prime}}(m_i || i)$ ;
9:     return  $\text{NI-PoE.Verify}(\eta_i, A_{s_m}, A_i, Q_i)$ 
10:  end if
11: end if
12: return 0

```

- $\text{Vallns}(B_i, B'_{s[l]}, l, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow (\{0, 1\})$. On input the block B_i to be inserted at position l , the updated block $B'_{s[l]}$ used to be at l , and $\mathcal{V}$'s local chain $\mathcal{C}$ with $\text{Len}(\mathcal{C}) = n$, the algorithm runs as follows, of which details are presented in Algorithm 4.

- 1) $\mathcal{V}$ checks entries like that in ValApp , except for additional checking of "chain" between B_i and $B'_{s[l]}$.
- 2) $\mathcal{V}$ outputs the verification result 0 or 1.

- $\text{Modify}(\mathcal{C}, \text{Len}(\mathcal{C}), m', l, (tk_{s[l]}, hk), \phi(N)) \rightarrow (\mathcal{C}')$. The inputs are $\mathcal{C}$ with $\text{Len}(\mathcal{C}) = n$, target Merkle root m' of the l -th block $B_{s[l]}$, chameleon hash key pair $(tk_{s[l]}, hk)$ for $B_{s[l]}$, and accumulator trapdoor $\phi(N)$. The algorithm executes as follows, of which the diagram is in Fig. 4 and the details are in Algorithm 5.

- 1) $\mathcal{M}$ retains previous hash and PoW nonce of the original $B_{s[l]}$, and obtains sequence number i' routinely.
- 2) $\mathcal{R}$ erases $H_{\text{prime}}(m_i || i)$ derived from the original $B_{s[l]}$ efficiently, from the accumulator state via $\phi(N)$.

Algorithm 3 Insert Block**Require:** $\mathcal{C}, \text{Len}(\mathcal{C}), m, ctr, l, (tk_i, tk_{s[l]}, hk)$.**Ensure:** $\mathcal{C}'$.

```

1: parse  $\mathcal{C}$  as  $(B_1, B_2, \dots, B_{s[n]})$ ;
2: parse  $B_{s[l]} := \langle p_{s[l]}, ctr_{s[l]}, m_{s[l]}, \vec{s}[l], \xi_{s[l]}, A_{s[l]}, w_{s[l]} \rangle$ ;
3:  $cont_{s[l]} \leftarrow p_{s[l]} || m_{s[l]} || \vec{s}[l] || A_{s[l]} || w_{s[l]}$ ;
4:  $h_{s[l]} \leftarrow \mathcal{CH}.\text{RHGen}(cont_{s[l]}, \xi_{s[l]}, hk)$ ;
5:  $s_m \leftarrow \max_{j \in [n]} \vec{s}[j], i \leftarrow s_m + 1, \{\text{sequence number}\}$ 
6:  $\eta_i \leftarrow H_{\text{prime}}(m || i), A_i \leftarrow A_{s_m}^{\eta_i}, \{\text{accumulator state}\}$ 
7:  $Q_i \leftarrow \text{NI-PoE.Prove}(\eta_i, A_{s_m}, A_i)$ ;
8:  $w_i \leftarrow Q_i, \{\text{witness of } A_i\}$ 
9:  $cont_i \leftarrow p_{s[l]} || m || i || A_i || w_i$ ;
10:  $(h_i, \xi_i) \leftarrow \mathcal{CH}.\text{HGen}(tk_i, cont_i)$ ;  $\{\text{checking string}\}$ 
11:  $B_i := \langle p_{s[l]}, ctr, m, i, \xi_i, A_i, w_i \rangle$ ;
12:  $p'_{s[l]} \leftarrow H_1(ctr || h_i)$ ;  $\{\text{front block of } B_{s[l]} \text{ changes}\}$ 
13:  $cont'_{s[l]} \leftarrow p'_{s[l]} || m_{s[l]} || \vec{s}[l] || A_{s[l]} || w_{s[l]}$ ;
14:  $\xi'_{s[l]} \leftarrow \mathcal{CH}.\text{Adapt}(tk_{s[l]}, h_{s[l]}, cont_{s[l]}, \xi_{s[l]}, cont'_{s[l]})$ ;
     $\{\text{adaptation}\}$ 
15:  $B'_{s[l]} := \langle p'_{s[l]}, ctr_{s[l]}, m_{s[l]}, \vec{s}[l], \xi'_{s[l]}, A_{s[l]}, w_{s[l]} \rangle$ ;
16:  $\mathcal{C}' \leftarrow \mathcal{C}^{[n-l+1]} || B_i || B'_{s[l]} || \mathcal{C}$ ;
17: return  $\mathcal{C}'$ 

```

- 3) $\mathcal{M}$ continues to update the accumulator state to $A_{i'}$ by adding $H_{\text{prime}}(m' || i')$ from the updated $B_{i'}$, and gives witnesses $w_{i'}$ for both the validity of $A_{i'}$ and the non-membership of all history versions of the modified block.
- 4) $\mathcal{R}$ adapts the checking string for $cont_{i'}$ using $tk_{s[l]}$.
- 5) $\mathcal{M}$ updates $\mathcal{C}$ to $\mathcal{C}' \leftarrow \mathcal{C}^{[n-l+1]} || B_{i'} || \mathcal{C}$.

- $\text{ValMod}(B_i, l, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow (\{0, 1\})$. On input the updated l -th block B_i and $\mathcal{V}$'s local chain $\mathcal{C}$ with $\text{Len}(\mathcal{C}) = n$, it runs as follows, and the details are shown in Algorithm 6.

- 1) $\mathcal{V}$ checks the validity of sequence number and PoW nonce. No verification for blockchain connectivity is

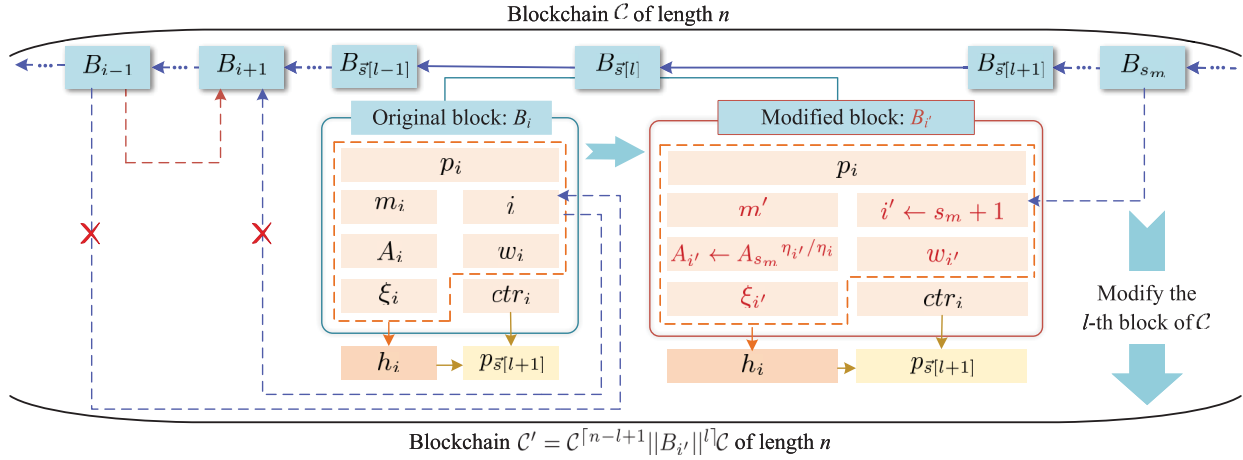


Fig. 4. The modify operation.

Algorithm 4 Validate Insert Operation**Require:** $B_i, B'_{s[l]}, l, C, \text{Len}(C)$.**Ensure:** $\{0, 1\}$.

- 1: parse $B_i := \langle p_i, ctr_i, m_i, i, \xi_i, A_i, w_i \rangle, w_i := Q_i$;
- 2: parse $B'_{s[l]} := \langle p'_{s[l]}, ctr'_{s[l]}, m'_{s[l]}, \tilde{s}[l]', \xi'_{s[l]}, A'_{s[l]}, w'_{s[l]} \rangle$;
- 3: parse $B_{s[l-1]} := \langle \cdot, ctr_{s[l-1]}, \dots \rangle$ in C ;
- 4: **if** block structure is valid **then**
- 5: $s_m \leftarrow \max_{j \in [n]} \tilde{s}[j]$ of C , parse $B_{s_m} := \langle \cdot, A_{s_m}, \cdot \rangle$;
- 6: calculate $h_i, h'_{s[l]}, h_{s[l-1]}$ by $\mathcal{CH}.\text{RHGen}(\cdot, \cdot, hk)$;
- 7: **if** $i == s_m + 1$ **and** $p_i == H_1(ctr_{s[l-1]} || h_{s[l-1]})$ **and**
 $H_1(ctr_i || h_i) < D$ **and** $p'_{s[l]} == H_1(ctr_i || h_i)$ **then**
- 8: $\eta_i \leftarrow H_{\text{prime}}(m_i || i)$;
- 9: **return** $\text{NI-PoE.Verify}(\eta_i, A_{s_m}, A_i, Q_i)$
- 10: **end if**
- 11: **end if**
- 12: **return** 0

needed since modification dose not impact front or behind blocks.

- 2) $\mathcal{V}$ verifies the validity of accumulator witnesses, concerning deletion of $B_{s[l]}$, addition of B_i and non-membership of its all history versions.
 - 3) $\mathcal{V}$ outputs the verification result 0/1 relying on the above verification results.
- Delete $(C, \text{Len}(C), L, m, ctr, (tk_{s_a}, tk_i, hk), \phi(N)) \rightarrow (C')$. The inputs are blockchain C with $\text{Len}(C) = n$, a set $L \subset [n]$ indicating consecutive blocks from $B_{s[L_{\min}]}$ to $B_{s[L_{\max}]}$ to be deleted, chameleon hash key pairs (tk_{s_a}, hk) for the $(L_{\max} + 1)$ -th block B_{s_a} and (tk_i, hk) for a new block B_i with Merkle root m and nonce ctr recording the deletion event to be appended to C , and accumulator trapdoor $\phi(N)$. Fig. 5 shows the schematic and Algorithm 7 presents the details.

- 1) $\mathcal{M}$ regards $\text{Head}(C)$ as front block of B_i if $n \notin L$, and the $(L_{\min} - 1)$ -th block B_{s_l} otherwise. Generate previous hash and sequence number of B_i as usual.
- 2) $\mathcal{R}$ deletes elements derived from blocks specified in L from the accumulator state efficiently. Here $\mathcal{U.A.Del}$ is executed via accumulator trapdoor

Algorithm 5 Modify Block**Require:** $C, \text{Len}(C), m', l, (tk_{s[l]}, hk), \phi(N)$.**Ensure:** C' .

- 1: parse $B_{s[l]} := \langle p_i, ctr_i, m_i, i, \xi_i, A_i, w_i \rangle$ in C ;
- 2: $cont_i \leftarrow p_i || m || i || A_i || w_i$;
- 3: $h_i \leftarrow \mathcal{CH}.\text{RHGen}(cont_i, \xi_i, hk)$;
- 4: $s_m \leftarrow \max_{j \in [n]} \tilde{s}[j], i' \leftarrow s_m + 1$; {sequence number}
- 5: initialize $\tilde{\eta}_{i'} \leftarrow \eta_i = H_{\text{prime}}(m_i || i)$;
- 6: $\tilde{\eta}_0 \leftarrow \prod_{j \in [n] \setminus \{l\}} H_{\text{prime}}(m_{s[j]} || \tilde{s}[j])$,
 $\tilde{A}_{i'} \leftarrow h^{\tilde{\eta}_0 \bmod \phi(N)}$; $\{\mathcal{R} \text{ deletes } \eta_i \text{ from accumulator}\}$
- 7: $Q_{i',1} \leftarrow \text{NI-PoE.Prove}(\eta_i, \tilde{A}_{i'}, A_{s_m})$; {witness for deletion of η_i from accumulator}
- 8: $\eta_{i'} \leftarrow H_{\text{prime}}(m' || i')$, $A_{i'} \leftarrow (\tilde{A}_{i'})^{\eta_{i'}}$; {accumulator state}
- 9: $Q_{i',2} \leftarrow \text{NI-PoE.Prove}(\eta_{i'}, \tilde{A}_{i'}, A_{i'})$; {witness for addition of $\eta_{i'}$ into accumulator}
- 10: $\tilde{\eta} \leftarrow \tilde{\eta}_0 \cdot \eta_{i'}$;
- 11: **if** $w_i == (\dots, \eta_i, \dots)$ **then**
- 12: $\tilde{\eta}_{i'} \leftarrow \tilde{\eta}_{i'} \cdot \eta_i$;
- 13: **end if**
- 14: $(\alpha, \beta) \leftarrow \text{Exgcd}(\tilde{\eta}_{i'}, \tilde{\eta}), \mu_{i'} \leftarrow h^\alpha, v_{i'} \leftarrow A_{i'}^\beta$;
- 15: $(z_{i'}, d_{i'}, Q_{i',3}) \leftarrow \text{NI-PoKE.Prove}(\beta, A_{i'}, v_{i'})$; {witness validity of non-membership witness}
- 16: $Q_{i',4} \leftarrow \text{NI-PoE.Prove}(\tilde{\eta}_{i'}, \mu_{i'}, h \cdot v_{i'}^{-1})$; {witness non-membership of all previous versions in B_i }
- 17: $w_{i'} \leftarrow (Q_{i',1}, \tilde{A}_{i'}, Q_{i',2}, z_{i'}, Q_{i',3}, d_{i'}, \tilde{\eta}_{i'}, \mu_{i'}, v_{i'}, Q_{i',4})$; {witness corresponding to $A_{i'}$ }
- 18: $cont_{i'} \leftarrow p_i || m' || i' || A_{i'} || w_{i'}$;
- 19: $\xi_{i'} \leftarrow \mathcal{CH}.\text{Adapt}(tk_{s[l]}, h_i, cont_i, \xi_i, cont_{i'})$; {adaptation}
- 20: $B_{i'} := \langle p_i, ctr_i, m', i', \xi_{i'}, A_{i'}, w_{i'} \rangle$;
- 21: $C' \leftarrow C^{[n-l+1] || B_{i'}} || C$;
- 22: **return** C'

$\phi(N)$ and by allowing inputting a set of elements rather than one to be deleted.

- 3) $\mathcal{M}$ continues to update the accumulator state by adding the information of B_i , and gives witness w_i for both validity of the new accumulator state and

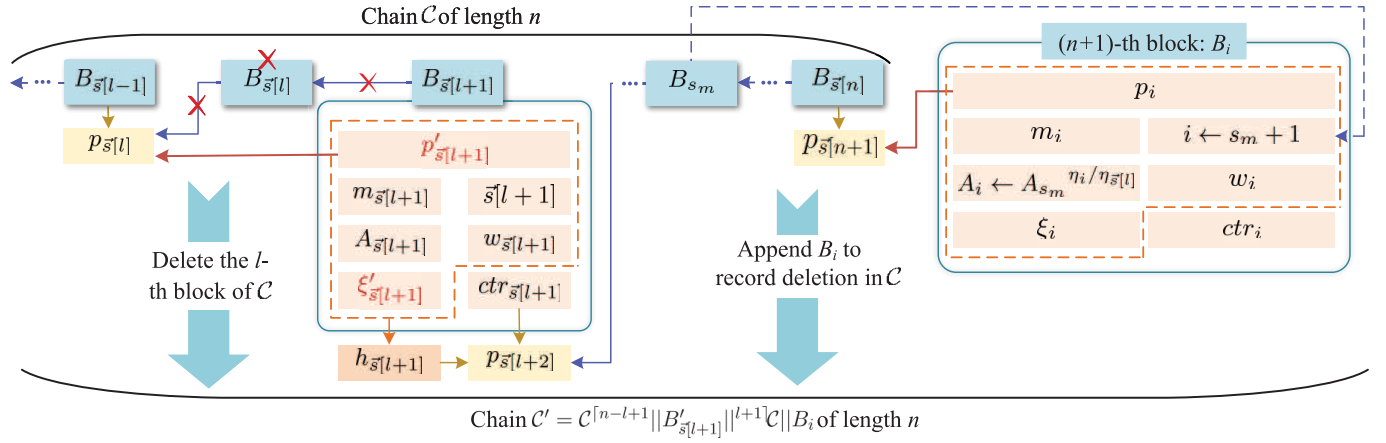


Fig. 5. The delete operation.

Algorithm 6 Validate Modify Operation**Require:** $B_i, \mathcal{C}, l, \text{Len}(\mathcal{C})$.**Ensure:** $\{0, 1\}$.

```

1: parse  $B_i := \langle p_i, ctr_i, m_i, i, \xi_i, A_i, w_i \rangle$ ,
    $w_i := (Q_{i,1}, \bar{A}_i, Q_{i,2}, z_i, Q_{i,3}, d_i, \eta_i, \mu_i, v_i, Q_{i,4})$ ;
2: parse  $B_{\bar{s}[l]} := \langle \dots, m_{\bar{s}[l]}, \bar{s}[l], \dots \rangle$  in  $\mathcal{C}$ ;
3: if block structure is valid then
4:    $s_m \leftarrow \max_{j \in [n]} \bar{s}[j]$  of  $\mathcal{C}$ , parse  $B_{s_m} := \langle \dots, A_{s_m}, \cdot \rangle$ ;
5:    $cont_i \leftarrow p_i || m_i || i || A_i || w_i$ ;
6:    $h_i \leftarrow \mathcal{CH}.\text{RHGen}(cont_i, \xi_i, hk)$ ;
7:   if  $H_1(ctr_i || h_i) < D$  and  $i == s_m + 1$  then
8:      $\eta_i \leftarrow H_{\text{prime}}(m_i || i)$ ,  $\eta_{\bar{s}[l]} \leftarrow H_{\text{prime}}(m_{\bar{s}[l]} || \bar{s}[l])$ ;
9:     return NI-PoE.Verify( $\eta_{\bar{s}[l]}, \bar{A}_i, A_{s_m}, Q_{i,1}$ ) and
       NI-PoE.Verify( $\eta_i, \bar{A}_i, A_i, Q_{i,2}$ ) and
       NI-PoKE.Verify( $A_i, v_i, z_i, d_i, Q_{i,3}$ ) and
       NI-PoE.Verify( $\eta_i, \mu_i, hv_i^{-1}, Q_{i,4}$ )
10:  end if
11: end if
12: return 0

```

non-membership of all history versions of deleted blocks.

- 4) $\mathcal{R}$ generates checking string ξ_i for B_i via tk_i and adapts ξ_{s_a} for B_{s_a} via tk_{s_a} .
- 5) $\mathcal{M}$ shrinks $\mathcal{C}$ to $\mathcal{C}' \leftarrow \mathcal{C}^{[n-L_{\min}+1]||B'_{s_a}||^{L_{\max}+1}} \mathcal{C} || B_i$ with $\text{Len}(\mathcal{C}') = n - |L| + 1$.

- ValDel($B_i, L, \mathcal{C}, \text{Len}(\mathcal{C})$) $\rightarrow \{0, 1\}$. On input the recording block B_i , position set $L \subset [n]$ of deleted blocks, and $\mathcal{V}$'s local chain $\mathcal{C}$ with $\text{Len}(\mathcal{C}) = n$, it runs as follows, and the details are in Algorithm 8.

- 1) $\mathcal{V}$ makes a routine verification of PoW nonce, sequence number and the connectivity of relevant blocks.
- 2) $\mathcal{V}$ verifies witnesses for delete operation, appending B_i , and invalidation of all deleted blocks' histories.
- 3) $\mathcal{V}$ outputs its verification decision 0 or 1.

Algorithm 7 Delete Block**Require:** $\mathcal{C}, \text{Len}(\mathcal{C}), L, m, ctr, (tk_{s_a}, tk_i, hk), \phi(N)$.**Ensure:** $\mathcal{C}'$.

```

1: parse  $B_{s_i}, B_{s_a}$  in  $\mathcal{C}$ ;
2:  $cont_{s_a} \leftarrow p_{s_a} || m_{s_a} || s_a || A_{s_a} || w_{s_a}$ ;
3:  $h_{s_a} \leftarrow \mathcal{CH}.\text{RHGen}(cont_{s_a}, \xi_{s_a}, hk)$ ;
4:  $p' \leftarrow H_1(ctr_{s_i} || h_{s_i})$ ,  $cont'_{s_a} \leftarrow p' || m_{s_a} || s_a || A_{s_a} || w_{s_a}$ ;
5:  $\xi' \leftarrow \mathcal{CH}.\text{Adapt}(tk_{s_a}, h_{s_a}, cont_{s_a}, \xi_{s_a}, cont'_{s_a})$ ;
6:  $B'_{s_a} := \langle p', ctr_{s_a}, m_{s_a}, s_a, \xi', A_{s_a}, w_{s_a} \rangle$ ;
7:  $s_m \leftarrow \max_{j \in [n]} \bar{s}[j]$ ,  $i \leftarrow s_m + 1$ ; {sequence number}
8:  $p_i \leftarrow H_1(ctr_{\bar{s}[n]} || h_{\bar{s}[n]})$ ;
9:  $\tilde{\eta}_0 \leftarrow \prod_{l \in [n] \setminus L} H_{\text{prime}}(m_{\bar{s}[l]} || \bar{s}[l])$ ,  $\bar{A}_i \leftarrow h^{\tilde{\eta}_0 \bmod \phi(N)}$ ;
   { $\mathcal{R}$  erases blocks indicated in  $L$  from accumulator}
10: initialize  $\hat{\eta}_1 \leftarrow 1, \hat{\eta}_2 \leftarrow 1$ ;
11: for  $l \in L$  do
12:    $\hat{\eta}_1 \leftarrow \hat{\eta}_1 \cdot H_{\text{prime}}(m_{\bar{s}[l]} || \bar{s}[l])$ ;
13:   if  $w_{\bar{s}[l]} == (\dots, \eta_{\bar{s}[l]}, \dots)$  then
14:      $\hat{\eta}_2 \leftarrow \hat{\eta}_2 \cdot \eta_{\bar{s}[l]}$ ;
15:   end if
16: end for
17:  $Q_{i,1} \leftarrow \text{NI-PoE.Prove}(\hat{\eta}_1, \bar{A}_i, A_{s_m})$ ; {witness for deletion of blocks indicated by  $L$ }
18:  $\eta_i \leftarrow H_{\text{prime}}(m || i)$ ,  $A_i \leftarrow \bar{A}_i^{\eta_i}$ ; {accumulator state}
19:  $Q_{i,2} \leftarrow \text{NI-PoE.Prove}(\eta_i, \bar{A}_i, A_i)$ ; {witness for addition of  $\eta_i$  in accumulator}
20:  $\hat{\eta}_i \leftarrow \hat{\eta}_1 \cdot \hat{\eta}_2$ ,  $\tilde{\eta} \leftarrow \tilde{\eta}_0 \cdot \eta_i$ ;
21:  $(\alpha, \beta) \leftarrow \text{Exgcd}(\hat{\eta}_i, \tilde{\eta})$ ,  $\mu_i \leftarrow h^\alpha$ ,  $v_i \leftarrow A_i^\beta$ ;
22:  $(z_i, d_i, Q_{i,3}) \leftarrow \text{NI-PoKE.Prove}(\beta, A_i, v_i)$ ; {witness validity of non-membership witness}
23:  $Q_{i,4} \leftarrow \text{NI-PoE.Prove}(\hat{\eta}_i, \mu_i, hv_i^{-1})$ ; {witness non-membership of all previous versions of deleted blocks}
24:  $w_i \leftarrow (Q_{i,1}, \bar{A}_i, Q_{i,2}, z_i, Q_{i,3}, d_i, \eta_i, \mu_i, v_i, Q_{i,4})$ ; {witness corresponding to  $A_i$ }
25:  $(h_i, \xi_i) \leftarrow \mathcal{CH}.\text{HGen}(tk_i, p_i || m || i || A_i || w_i)$ ; {checking string}
26:  $B_i := \langle p_i, ctr, m, i, \xi_i, A_i, w_i \rangle$ ;
27:  $\mathcal{C}' \leftarrow \mathcal{C}^{[n-L_{\min}+1]||B'_{s_a}||^{L_{\max}+1}} \mathcal{C} || B_i$ ;
28: return  $\mathcal{C}'$ 

```

Algorithm 8 Validate Delete Operation**Require:** $B_i, L, \mathcal{C}, \text{Len}(\mathcal{C})$.**Ensure:** $\{0, 1\}$.

```

1: parse  $B_i := \langle p_i, ctr_i, m_i, i, \xi_i, A_i, w_i \rangle$ ,
    $w_i := (Q_{i,1}, A_i, Q_{i,2}, z_i, Q_{i,3}, d_i, \eta_i, \mu_i, v_i, Q_{i,4})$ ;
2: parse  $B_{\vec{s}[L_{min}-1]} := \langle \dots, s_i, \dots \rangle$ ,  $B_{\vec{s}[L_{max}+1]} := \langle \dots, s_a, \dots \rangle$ ,  $B_{\vec{s}[n]} := \langle \dots, \vec{s}[n], \dots \rangle$  in  $\mathcal{C}$ ;
3: if block structure is valid then
4:    $s_m \leftarrow \max_{j \in [n]} \vec{s}[j]$  of  $\mathcal{C}$ , parse  $B_{s_m} := \langle \dots, A_{s_m}, \cdot \rangle$ ;
5:    $h_i, h_{s_i}, h_{s_a}, h_{\vec{s}[n]}$  by  $\mathcal{CH}.\text{RHGen}(\cdot, \cdot, hk)$ ;
6:   if  $H_1(ctr_i || h_i) < D$  and  $i == s_m + 1$  and  $p_i == H_1(ctr_{\vec{s}[n]} || h_{\vec{s}[n]})$  and  $p_{s_a} == H_1(ctr_{s_i} || h_{s_i})$  then
7:      $\eta_i \leftarrow H_{\text{prime}}(m_i || i)$ ,  $\eta_1 \leftarrow \prod_{l \in L} H_{\text{prime}}(m_{\vec{s}[l]} || \vec{s}[l])$ ;
8:     return NI-PoE.Verify( $\eta_1, \bar{A}_i, A_{s_m}, Q_{i,1}$ ) and
       NI-PoE.Verify( $\eta_i, \bar{A}_i, A_i, Q_{i,2}$ ) and
       NI-PoKE.Verify( $A_i, v_i, z_i, d_i, Q_{i,3}$ ) and
       NI-PoE.Verify( $\eta_i, \mu_i, h v_i^{-1}, Q_{i,4}$ )
9:   end if
10: end if
11: return 0

```

Algorithm 9 Validate Blockchain**Require:** $\mathcal{C}, \text{Len}(\mathcal{C})$.**Ensure:** $\{0, 1\}$.

```

1: parse  $\mathcal{C}$  as  $(B_1, B_2, \dots, B_{\vec{s}[n]})$ ;
2: for  $j \in [n]$  do
3:    $cont_{\vec{s}[j]} \leftarrow p_{\vec{s}[j]} || m_{\vec{s}[j]} || \vec{s}[j] || A_{\vec{s}[j]} || w_{\vec{s}[j]}$ ;
4:    $h_{\vec{s}[j]} \leftarrow \mathcal{CH}.\text{RHGen}(cont_{\vec{s}[j]}, \xi_{\vec{s}[j]}, hk)$ ;
5:   if  $H_1(ctr_{\vec{s}[j]} || h_{\vec{s}[j]}) \geq D$  or  $H_1(ctr_{\vec{s}[j-1]} || h_{\vec{s}[j-1]}) \neq p_{\vec{s}[j]}$  then
6:     return 0
7:   end if
8: end for
9:  $\tilde{\eta} \leftarrow \prod_{j \in [n]} H_{\text{prime}}(m_{\vec{s}[j]} || \vec{s}[j])$ ;
10:  $s_m \leftarrow \max_{j \in [n]} \vec{s}[j]$  of  $\mathcal{C}$ , parse  $B_{s_m} := \langle \dots, A_{s_m}, \cdot \rangle$ ;
11: if  $A_{s_m} \neq h^{\tilde{\eta}}$  then {verification of accumulator state}
12:   return 0
13: end if
14: return 1

```

- ValChain($\mathcal{C}, \text{Len}(\mathcal{C})$) $\rightarrow (\{0, 1\})$. On input $\mathcal{C}$ with $\text{Len}(\mathcal{C}) = n$, $\mathcal{V}$ verifies as follows, of which details are described in Algorithm 9.

- 1) Check connectivity of successive blocks and validity of PoW nonce of all blocks in $\mathcal{C}$.
- 2) Obtain the latest accumulator state in $\mathcal{C}$, and check whether it strictly contains all existing blocks.
- 3) Output 1 if all the above validations pass.

Note that, in Delete, blocks indicated by multiple sets can be deleted in batch mode, on condition that the connectivity of front and behind blocks is maintained well, which can be derived easily and is omitted here for simplicity.

Remark 1: According to the above description, we can see that $\mathcal{R}$ is a central authority responsible for holding the

trapdoor key of the chameleon hash family and providing a trusted setup for the trapdoorless universal accumulator.

However, such power possessed by $\mathcal{R}$ can be controlled in two ways, as is summarized in [33], where either 1) it may be available to some fully trusted single party (like $\mathcal{R}$) in a permissioned setting, or 2) the trapdoor key is generated using MPC and the accumulator is also setup in a distributed way by some set of parties.

Hence, the proposed blockchain can be adjusted to the permissionless setting in theory similar to [11], where there is no central trusted authority. The main idea is to have the chameleon hash family and the trapdoorless universal accumulator be secretly setup among some set of users that are in charge of redacting the blockchain. Such a set of users can be found among users with higher levels of trustworthiness and contribution in the blockchain, like full nodes in Bitcoin, which are believed and accepted by the rest lightweight users forming the majority. However, finding such a set of users is a challenging task, which needs some specifically designed consensus mechanism and is out of the scope of this paper. Since we focus on how to equip redactable blockchains with verifiability and fully editability in this paper, which is independent of which of the above two approaches is going to be used. Thus, we presented the blockchain by introducing the fully trusted $\mathcal{R}$ for simplicity, and the extension to the permissionless setting is omitted here.

V. SECURITY ANALYSIS

We analyze the security of proposed verifiable and redactable blockchain, involving block security, chain sequence growth, chain sequence quality, and editable common presequence.

Theorem 1: If the chameleon hash function $\mathcal{CH}$ satisfies both collision resistance and key-exposure freeness, then the verifiable and redactable blockchain with fully editing operations satisfies the propriety of block security.

Proof: Suppose that there exists a PPT adversary $\mathcal{A}$ against the block security, then we can construct a PPT algorithm $\mathcal{B}$ breaking collision resistance or key-exposure freeness of chameleon hash functions.

To succeed, $\mathcal{A}$ should output a block $B^* = (p_i, ctr_i, m^*, i, \xi^*, A^*, w^*)$ at the challenge phase, such that at least one of the following events happen with non-negligible probability:

- 1) $E_1 : \text{ValApp}(B^*, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow 1$.
- 2) $E_2 : \text{ValIns}(B^*, B'_{\vec{s}[l]}, l, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow 1$.
- 3) $E_3 : \text{ValMod}(B^*, l, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow 1$.
- 4) $E_4 : \text{ValDel}(B^*, L, \mathcal{C}, \text{Len}(\mathcal{C})) \rightarrow 1$.

We claim that if any of these events occurs with non-negligible probability, the underlying hardness assumptions are broken with overwhelming probability.

We start with the situation when E_1 happens, which implies the simultaneous satisfaction of the following conditions:

$$\begin{cases} i == \max_{j \in \text{Len}(\mathcal{C})} \vec{s}[j] + 1 \\ H_1(ctr_i || h_i) < D \\ \text{NI-PoE.Verify}(\eta^*, A_{s_m}, A^*, Q^*) \rightarrow 1, \end{cases}$$

where $h_i \leftarrow \mathcal{CH}.\text{RHGen}(cont^*, \xi^*, hk)$, $cont^* \leftarrow p_i || m^* || i || A^* || w^*$ and $\eta^* \leftarrow H_{\text{prime}}(m^* || i)$.

Observe that without the attack launched by $\mathcal{A}$, the exact message concerning B_i is $cont_i = p_i || m_i || i || A_i || w_i$, and that the corresponding checking string fed back from $\mathcal{R}$ is ξ_i . Obviously, there are $h_i = \mathcal{CH}.\text{RHGen}(cont_i, \xi_i, hk)$ and ctr_i satisfying $H_1(ctr_i || h_i) < D$. If $h_i = \mathcal{CH}.\text{RHGen}(cont^*, \xi^*, hk)$ holds for B^* , collision resistance of $\mathcal{CH}$ is broken by $\mathcal{B}$, which happens with negligible probability [28]. Alternatively, if $\mathcal{A}$ reuses a collision correctly generated by $\mathcal{B}$, a valid forgery implies that the sequence number equals $\max_{j \in \text{Len}(C)} \bar{s}[j] + 1$. However, for the fact that the reused collision is calculated before the current round, the forgery succeeds with negligible probability.

Since E_2 is similar to E_1 besides the operation position, the analysis can be easily derived from the above and the probability is negligible as well.

Verification conditions of E_3 are

$$\begin{cases} i == \max_{j \in \text{Len}(C)} \bar{s}[j] + 1 \\ H_1(ctr_i || h_i) < D \\ \text{NI-PoE.Verify}(\eta^*, \bar{A}_i, A^*, Q_{i,2}), \end{cases}$$

where $\bar{A}_i^{\eta_{\bar{s}[L]}} = A_{s_m}$ and $\eta_{\bar{s}[L]} \leftarrow H_{\text{prime}}(m_{\bar{s}[L]} || \bar{s}[L])$ are generated from $B_{\bar{s}[L]} = (p_i, ctr_i, m_{\bar{s}[L]}, \bar{s}[L], \xi_{\bar{s}[L]}, A_{\bar{s}[L]}, w_{\bar{s}[L]})$ before modification.

We observe that $\mathcal{A}$ already holds at least two collisions of h_i before forgery, namely $(cont_{\bar{s}[L]}, \xi_{\bar{s}[L]})$ and $(cont_i, \xi_i)$, correctly generated by $\mathcal{R}$. If $\mathcal{A}$ outputs another collision $(cont^*, \xi^*)$ of h_i to make B^* pass validation with non-negligible probability, then $\mathcal{B}$ breaks key-exposure freeness property of $\mathcal{CH}$ with $\mathcal{A}$'s ability, which is proved infeasible in [28] however. Besides, the alternative that $\mathcal{A}$ replays $(cont^*, \xi^*)$ correctly generated by $\mathcal{R}$ is infeasible, of which the analysis is similar to that of E_1 .

Finally, we analyze the occurrence of E_4 , which is similar to that of E_3 , except for the position of newly generated recording block. Thus, we conclude that the probability of E_4 is negligible. $\square$

Recall that the proposed verifiable and redactable blockchain is a transformation of conventional blockchains, which modifies the consensus to largest sequence number principle, enables the verification of blockchain state, etc. Inspired by the security analysis in [16] and [32], we prove that an immutable blockchain Γ satisfies chain sequence growth, chain sequence quality and editable common presequence first, and then demonstrate that our verifiable and redactable blockchain Γ' preserves these properties aforementioned.

We begin by introducing some relevant parameters:

- n : the total number of users in the blockchain.
- t : the number of users controlled by $\mathcal{A}$.
- q_r : the maximum number of queries submitted by a user to compete in a round.
- p : the success probability of a single query.
- f : the probability that at least one honest user succeeds in competing in a round, and there is

$$f = 1 - (1 - p)^{q_r(n-t)} \geq \frac{pq_r(n-t)}{1 + pq_r(n-t)}.$$

- ϵ : the quality of concentration of random variables in executions.
- δ : the advantage of honest users ($\delta \geq 2f + 2\epsilon$), and it is obvious that $\frac{t}{n-t} \leq 1 - \delta$.

For each round ro_i , $j \in [q_r]$ and $k \in [t]$, there are three Boolean random variables $\dot{X}_{ro_i}$, $\dot{X}_{ro_i}$ and $\dot{X}_{ro_i j k}$. If an honest user wins the competition in round ro_i , then $\dot{X}_{ro_i} = 1$ and ro_i is a successful round, otherwise $\dot{X}_{ro_i} = 0$. If exactly one honest user wins, then $\dot{X}_{ro_i} = 1$ and ro_i is called a *uniquely* successful round, otherwise $\dot{X}_{ro_i} = 0$. If the k -th corrupted user wins via the j -th query in ro_i , then $\dot{X}_{ro_i j k} = 1$, and $\dot{X}_{ro_i j k} = 0$ otherwise.

Theorem 2: If Γ satisfies (τ, x) -chain sequence growth, then the proposed Γ' satisfies (τ, x) -chain sequence growth as well, where the speed coefficient is $\tau = (1 - \epsilon)f$.

Proof: First, we prove that Γ satisfies (τ, x) -chain sequence growth by induction on $x \geq 0$. For the basis ($ro_1 = ro_2$), if an honest user has a chain $\mathcal{C}_1$ with the largest sequence number s_{max} at ro_1 , then $\mathcal{C}_1$ must have been broadcast at round earlier than ro_1 , which is followed by that all honest users received $\mathcal{C}_1$ by ro_1 .

For the inductive step, every honest user will receive a chain with the largest sequence number at least $s'_{max} = s_{max} + \sum_{ro_i=ro_1}^{ro_2-2} \dot{X}_{ro_i}$ by round $(ro_2 - 1)$ by the inductive hypothesis. When $\dot{X}_{ro_2} = 0$, the statement follows directly. Assume $\dot{X}_{ro_2} = 1$, observe that every honest user queries with a blockchain of the largest sequence number at least s'_{max} at round $(ro_2 - 1)$. It follows that successful honest users at round $(ro_2 - 1)$ broadcasts a blockchain with the largest sequence number $s'_{max} + 1 = s_{max} + \sum_{ro_i=ro_1}^{ro_2-1} \dot{X}_{ro_i}$.

Since the expectation of $\dot{X}_{ro_i}$ is $\mathbb{E}[\dot{X}_{ro_i}] = f$, there is $\mathbb{E}[\sum_{ro_i=ro_1}^{ro_2-1} \dot{X}_{ro_i}] = xf$. In a typical execution, the bound for $\sum_{ro_i=ro_1}^{ro_2-1} \dot{X}_{ro_i}$ is $(1 - \epsilon)\mathbb{E}[\sum_{ro_i=ro_1}^{ro_2-1} \dot{X}_{ro_i}] < \sum_{ro_i=ro_1}^{ro_2-1} \dot{X}_{ro_i} < (1 + \epsilon)\mathbb{E}[\sum_{ro_i=ro_1}^{ro_2-1} \dot{X}_{ro_i}]$, so there is

$$(1 - \epsilon)fx < \sum_{ro_i=ro_1}^{ro_2-1} \dot{X}_{ro_i},$$

i.e., the speed coefficient is $\tau = (1 - \epsilon)f$. This completes the proof that Γ satisfies (τ, x) -chain sequence growth, where $\tau = (1 - \epsilon)f$.

Second, we consider the situation when it comes fully editing operations, i.e., when Γ is extended to Γ' . Note that the sequence number always increases by 1 after each operation. Therefore, we conclude that Γ' satisfies (τ, x) -chain sequence growth whenever Γ satisfies (τ, x) -chain sequence growth, and this completes the proof. $\square$

Theorem 3: If Γ satisfies (γ, y) -chain sequence quality, then the proposed Γ' satisfies (γ, y) -chain sequence quality, where the quality coefficient $\gamma = (1 + \frac{\delta}{2}) \frac{t}{n-t} < 1 - \frac{\delta}{2}$.

Proof: First, we prove that Γ satisfies (γ, y) -chain sequence quality. Denote B_i as the block with sequence number i in Γ possessed by an honest user at some round ro , and consider some L_1 blocks with consecutive sequence numbers: $B_u, \dots, B_v$. Define L_2 as the least number of blocks with consecutive sequence numbers that include the L_1 given

ones: $B_{u'}, \dots, B_{v'}$, which implies $u' \leq u$ and $v \leq v'$. Also, these blocks have two properties: 1) $B_{u'}$ was created by an honest user or is B_1 if such a block does not exist, 2) there exists a round at which an honest user was trying to extend the blockchain with the largest sequence number being v' . Define also ro_1 as the round that $B_{u'}$ was created ($ro_1 = 0$ if $B_{u'}$ is the genesis block) and ro_2 as the first round that an honest user attempts to extend the blockchain with the largest sequence number being v' , and let $R = \{ro : ro_1 \leq ro < ro_2\}$ with $|R| = y$. Let L_0 be the number of blocks from honest users that are included in the L_1 blocks, towards a contradiction, we assume that

$$L_0 \leq L_1 \left[1 - \left(1 + \frac{\delta}{2} \right) \frac{t}{n-t} \right] \leq L_2 \left[1 - \left(1 + \frac{\delta}{2} \right) \frac{t}{n-t} \right].$$

Suppose that all the L_2 blocks have been computed during rounds in R . Then, according to the definition of L_0 , L_1 and L_2 , as well as the above inequality, there is

$$\begin{aligned} \ddot{X}(R) &\geq L_2 - L_0 \geq L_2 \left(1 + \frac{\delta}{2} \right) \frac{t}{n-t} \\ &\geq \dot{X}(R) \left(1 + \frac{\delta}{2} \right) \frac{t}{n-t}. \end{aligned}$$

Recall $f \geq \frac{pq_r(n-t)}{1+pq_r(n-t)}$, the expectation of $\ddot{X}_{ro_i}$ is

$$\mathbb{E}[\ddot{X}_{ro_i}] = pq_r t = \frac{t}{n-t} pq_r(n-t) \leq \frac{t}{n-t} \cdot \frac{f}{1-f}.$$

In a typical execution, the bound for $\ddot{X}(R)$ is $\ddot{X}(R) < (1 + \epsilon)\mathbb{E}[\ddot{X}(R)]$, so there is

$$\ddot{X}(R) < (1 + \epsilon)|R| \frac{t}{n-t} \cdot \frac{f}{1-f} \leq \frac{(1 + \epsilon)(1 - \delta)f|R|}{1 - f}.$$

According to the above bound and $(1 - \epsilon)f|R| < \dot{X}(R) < (1 + \epsilon)f|R|$ in the proof of **Theorem 2**, there is

$$\ddot{X}(R) < \left(1 + \frac{\delta}{2} \right) \dot{X}(R) \frac{t}{n-t} < \left(1 - \frac{\delta}{2} \right) \dot{X}(R).$$

Thus, it contradicts $\ddot{X}(R) \geq \dot{X}(R) \left(1 + \frac{\delta}{2} \right) \frac{t}{n-t}$. This completes the proof that Γ satisfies (γ, y) -chain sequence quality, where the quality coefficient is $\gamma = \left(1 + \frac{\delta}{2} \right) \frac{t}{n-t} < 1 - \frac{\delta}{2}$.

Second, we consider that Γ is extended to Γ' . Though Γ' supports fully editing operations, the sequence numbers of blocks generated during a round are successive. An adversary $\mathcal{A}$ attempts to edit honest B into malicious B^* , by containing illegal content, to increase the fraction of adversarial blocks in Γ' , breaking chain sequence quality. However, as is proved in **Theorem 1**, $\mathcal{A}$ would succeed with negligible probability because only honest blocks is incorporated to Γ' . This completes the proof. $\square$

Theorem 4: If Γ satisfies κ -editable common presequence, then the proposed Γ' satisfies κ -editable common presequence.

Proof: First, we prove that Γ satisfies κ -editable common presequence by using Reduction to Absurdity. Consider blockchains $\mathcal{C}_1$ and $\mathcal{C}_2$, which are possessed by two honest users at ro_1 and ro_2 separately, in violation of the editable common presequence property, i.e., blocks with sequence

number no larger than $\max_{j \in [\text{Len}(\mathcal{C}_1)]} \vec{s}[j] - \kappa$ in $\mathcal{C}_1$ do not compose the presequence of $\mathcal{C}_2$, denoted by $\mathcal{C}_1 \xrightarrow{\text{seq}} -\kappa \not\prec \mathcal{C}_2$ (otherwise $\mathcal{C}_1 \xrightarrow{\text{seq}} -\kappa < \mathcal{C}_2$). Let ro be the smallest round satisfying $ro_1 \leq ro \leq ro_2$, where an honest user adopts $\mathcal{C}_2'$ such that $\mathcal{C}_1 \xrightarrow{\text{seq}} -\kappa \not\prec \mathcal{C}_2'$. Observe that by our assumptions such ro is well defined, e.g., ro_2 is such a round though not necessarily the smallest one.

We consider two cases. In case $ro = ro_1$, we deduce that at round ro , two honest users have adopted $\mathcal{C}_1$ and $\mathcal{C}_2'$ separately, for which it holds $\mathcal{C}_1 \xrightarrow{\text{seq}} -\kappa \not\prec \mathcal{C}_2'$. Given that $\mathcal{C}_2'$ is adopted at ro by an honest user and $\mathcal{C}_1$ is the blockchain possessed by an honest user in the same round, it holds that $\max_{j \in [\text{Len}(\mathcal{C}_1)]} \vec{s}[j] \leq \max_{j \in [\text{Len}(\mathcal{C}_2')] } \vec{s}[j]$ due to the largest sequence number principle. Consider the fact that at the same round, $\mathcal{C}_1$ is adopted by an honest user and $\mathcal{C}_2'$ is either adopted or diffused by an honest user, there are $\mathcal{C}_1 \xrightarrow{\text{seq}} -\kappa < \mathcal{C}_2'$ and $\mathcal{C}_2' \xrightarrow{\text{seq}} -\kappa < \mathcal{C}_1$, we obtain a contradiction.

Consider the case $ro_1 < ro$. For the honest user that adopted $\mathcal{C}_2'$, let $\mathcal{C}_1'$ be the blockchain he/she adopted at round $(ro - 1)$. Then there are $\mathcal{C}_1' \xrightarrow{\text{seq}} -\kappa < \mathcal{C}_2'$ and $\mathcal{C}_2' \xrightarrow{\text{seq}} -\kappa < \mathcal{C}_1'$. We claim that

$$\begin{aligned} &\left(\mathcal{C}_2' \xrightarrow{\text{seq}} -\kappa < \mathcal{C}_1' \right) \wedge \left(\mathcal{C}_1 \xrightarrow{\text{seq}} -\kappa < \mathcal{C}_1' \right) \\ &\wedge \left(\max_{j \in [\text{Len}(\mathcal{C}_2' \xrightarrow{\text{seq}} -\kappa)]} \vec{s}[j] \geq \max_{j \in [\text{Len}(\mathcal{C}_1 \xrightarrow{\text{seq}} -\kappa)]} \vec{s}[j] \right) \\ &\Rightarrow \mathcal{C}_1 \xrightarrow{\text{seq}} -\kappa < \mathcal{C}_2' \xrightarrow{\text{seq}} -\kappa, \end{aligned}$$

which implies directly that $\mathcal{C}_1 \xrightarrow{\text{seq}} -\kappa < \mathcal{C}_2'$ and contradicts the definition of ro . This completes the proof that Γ satisfies κ -editable common presequence.

Second, we consider the situation when Γ is extended to Γ' . When insert operation is supported, Γ' directly satisfies such a property concerning sequence numbers. However, in case of modify and delete operations, some blocks in the blockchain at ro_1 would disappear at ro_2 . Consider an adversary $\mathcal{A}$ proposing B^* to replace one block (the modification scenario) or several blocks (the deletion scenario). Observe that if such a block is malicious, the operation would not be validated by honest users. Therefore, the disappearance of blocks at ro_2 must result from authorized modify or delete operations. This concludes the proof. $\square$

How these properties play together: Through above properties, we indicate that Γ' is a live and persistent blockchain, which is immutable against edits unauthorized by $\mathcal{R}$.

VI. PERFORMANCE ANALYSIS

We implement the primary functionalities of the proposed blockchain based on Bitcoin, since Bitcoin is a typical blockchain and its inner design is widely accepted. To evaluate performance, the experimental results are compared with that of [11] supporting modification and deletion, as well as un-editable Bitcoin.

Similar to [11], we set the PoW difficulty target to be 0, so that the accurate runtime of each operation is measured

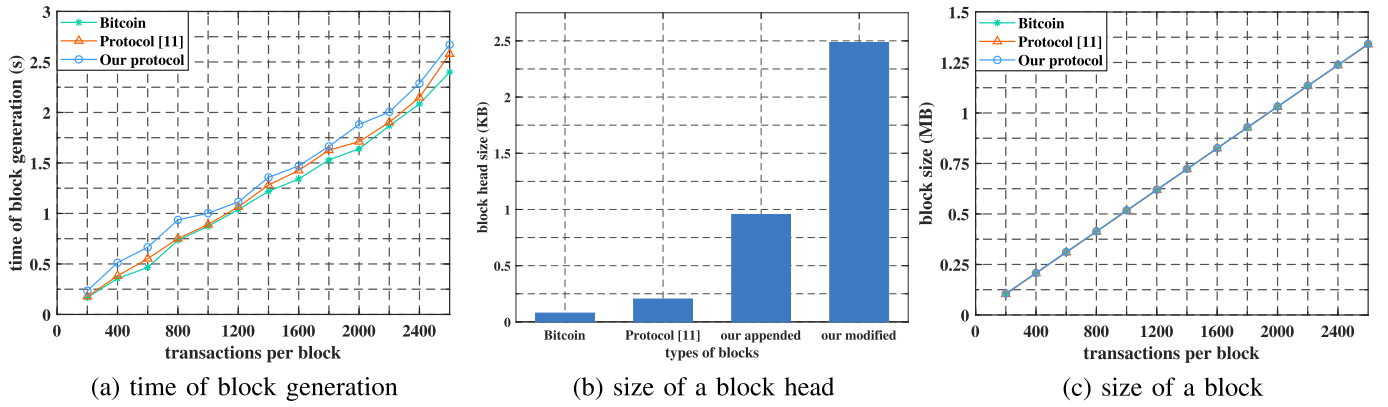


Fig. 6. Time cost and storage overhead of generating a block.

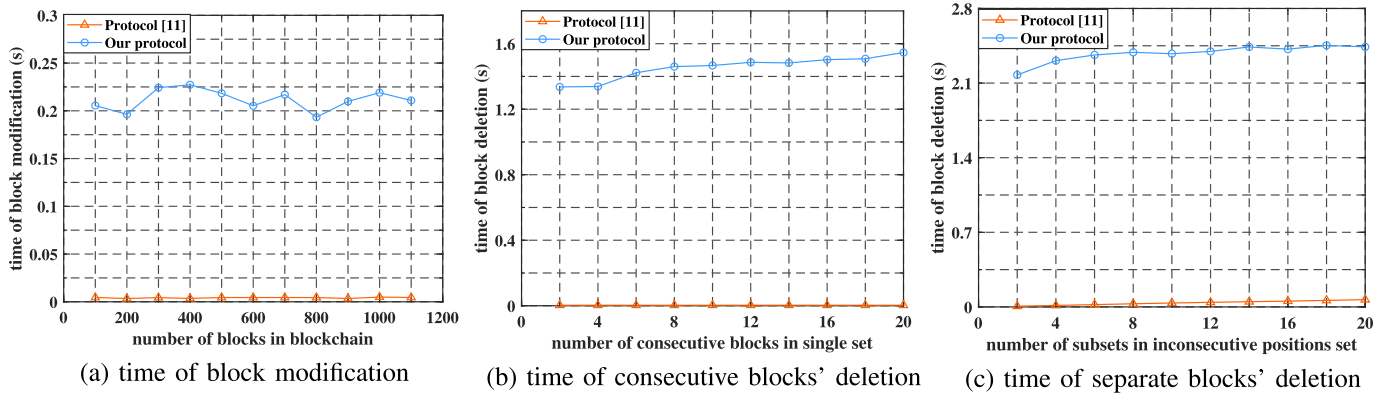


Fig. 7. Time cost of modifying and deleting blocks.

without the influence brought by solving PoW puzzles. Besides, the impact of communication is not considered in the test of each functionality and all transactions considered are of type `Pay-to-PubkeyHash`. Experiments are conducted on Ubuntu 20.04.4 LTS (2 GB memory) working on VMware 12.5.2 running in a PC equipped with Intel(R) Core(TM) i5-7500 CPU @ 3.40GHz and a 8GB RAM. The double trapdoor chameleon hash family is implemented with the P-256 curve and the trapdoorless universal accumulator is implemented with the key size of 3072 bits, which both offer 128-bit security level. Programs are executed in Python 3.8.10.

We demonstrate that our blockchain is practical from three aspects. The first is how much the block generation overhead is, relative to [11] and Bitcoin, when no editing operation occurs. The second is how long it takes to edit (modify/delete) an existing block, compared with [11]. And the final is cost of validating various operations as well as the whole blockchain.

A. Generate a Block

We test the cost of generating a block in Fig. 6. Note that, the block generation in our blockchain contains not only appending a block like [11] and Bitcoin, but also inserting one at any position in the blockchain. For simplicity, however, we do not distinguish these two operations, since the difference between them is just one call of `CH.Adapt`, which is negligible for both operations. From Fig. 6(a), we can see that the time

required to create blocks in Bitcoin is the least, for lacking of redactability. Besides, our runtime is slightly higher than that of [11], because we run `U.A.Add` and `U.A.MWit` additionally to realize verifiability of blockchain state. In Fig. 6(b) we show a comparison of the size of block heads in these three blockchains. Since we need to record membership witness of block's current content and non-membership witnesses of history versions, the size of block head is slightly bigger than the other two. However, such a storage cost difference is negligible for the whole block, as is shown in Fig. 6(c). Overall, the additional overhead in the process of block generation is small enough, which does not affect the efficiency of our verifiable and redactable blockchain.

B. Edit Blocks

Comparisons of the runtime to modify and delete blocks in our blockchain versus [11] are presented in Fig. 7. From Fig. 7(a), we can see that the computational cost of blockchain modification in this paper and [11] are both constant, while ours is slightly higher. This is because we call `U.A.Del` to invalidate history version of modified blocks to resist reversion attacks. Delete operation is tested in Fig. 7(b)-(c), where set L is divided into two categories: one containing consecutive positions, and the other containing inconsecutive positions. The latter can be regarded as a set containing multiple subsets of consecutive positions. Since one chameleon hash adaptation is needed behind each deleted subset in [11], the runtime remains

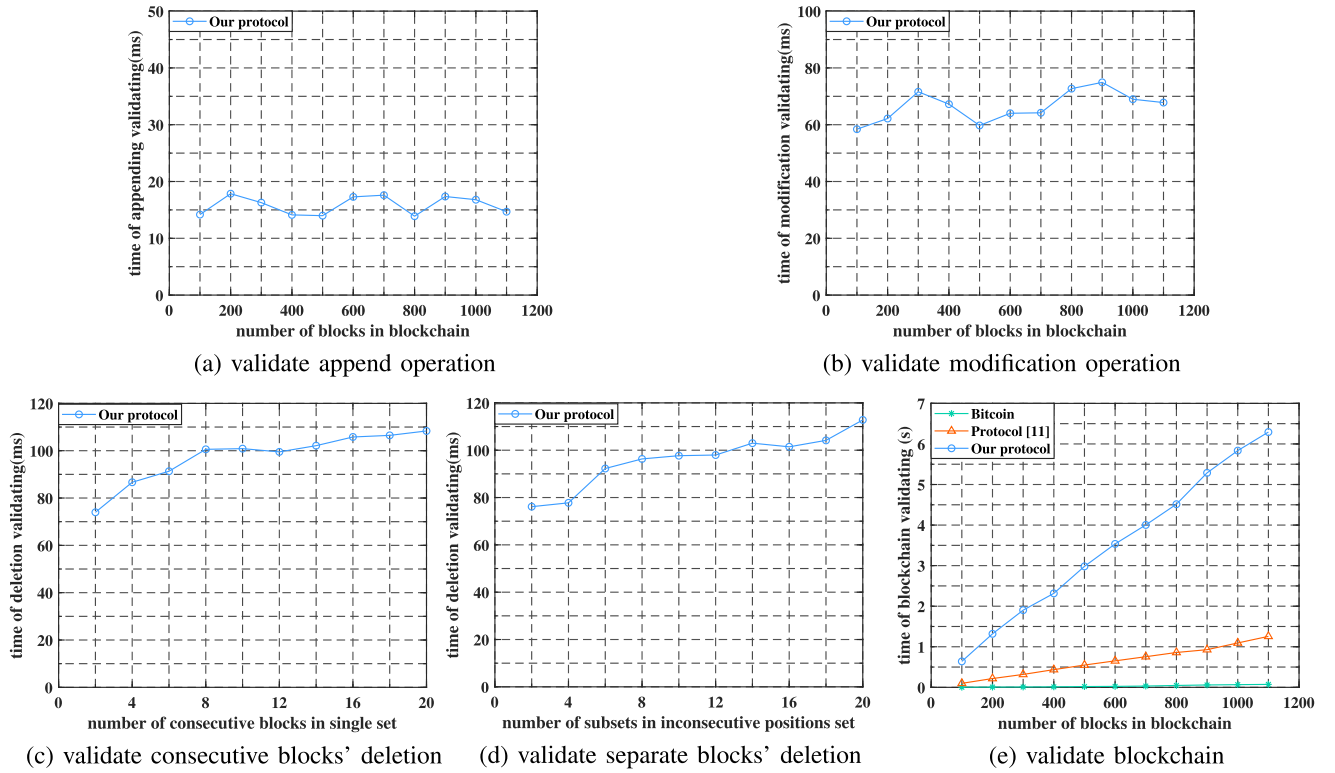


Fig. 8. Time cost of validating dynamic operations and the blockchain.

unchanged with the number consecutive blocks in a set while increases with the number of subsets. In our blockchain, except for $\mathcal{CH}.\text{Adapt}$, we need to compute $H_{\text{prime}}(\cdot)$ for all deleted blocks and multiply them together. As a consequence, the run time of deletion increases in both categories. However, since the additional cost is for realizing the verifiability of blockchain state, it is acceptable in practice.

C. Validate Editing Operations and Blockchain

Since the proposed redactable blockchain is verifiable, we test the overhead of validating each operation in Fig. 8. It is obvious in Fig. 8(a)-(b) that the validation time of block creation and modification does not change with the length of blockchain, costing around tens of milliseconds throughout, which is user friendly. The runtime of validating block deletion in two categories is tested in Fig. 8(c)-(d), separately. Since $H_{\text{prime}}(\cdot)$ of all deleted blocks should be computed and multiplied together, the runtime increases with both the number of subsets and the size of L . Note that such a runtime is just in millisecond grade. In Fig. 8(e), we compare how long it takes to validate a blockchain with length ranging from 100 to 1100 with [11] and Bitcoin. Although the validating time of all three blockchains increases as the length of the blockchain grows, it takes longer in the proposed blockchain than the other two. This is because current versions of all existing blocks should be collected to compare with the latest accumulator state, which can resist reversion attacks seriously affecting [11]. Besides, it is worth noting that users who download and validate the entire blockchain are typically volunteers with spare computation and bandwidth dedicated to maintaining the security and stability of the blockchain. Alternatively, they may be solo miners who

have substantially invested in high-performance equipment and seek significant exclusive profits. Therefore, in practice, the efficiency of blockchain validation can be further improved and would not be a significant computational burden.

In summary, the additional overhead of the proposed redactable blockchain in generating and editing blocks, as well as validating these operations, is minor and seems acceptable, without a noticeable impact on efficiency and availability. While the additional cost brought by validating the blockchain state is not negligible, it appears to be an acceptable trade-off for maintaining the security and stability of the redactable blockchain or gaining greater profits. Nevertheless, finding a more efficient construction of accumulators that shorten the runtime of blockchain validation is an important future work.

VII. CONCLUSION

We propose a verifiable and redactable blockchain supporting fully editing operations, enabling redactions of block objects and validation of blockchain state simultaneously for the first time, which has broad application prospects. Extensive experiments are conducted to show that these features in such a blockchain are achieved with seemingly acceptable overhead compared with the related work. In addition, a comprehensive analysis is presented to demonstrate the security in aspects of an individual block and the whole blockchain.

REFERENCES

- [1] S. Nakamoto. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [2] M. Pilkington, "Blockchain technology: Principles and applications," in *Research Handbook on Digital Transformations*. Cheltenham, U.K.: Edward Elgar Publishing, 2016, pp. 225–253.

- [3] R. Zhang, R. Xue, and L. Liu, "Security and privacy on blockchain," *ACM Comput. Surv.*, vol. 52, no. 3, pp. 1–34, 2019.
- [4] M. Belotti, N. Božić, G. Pujolle, and S. Secci, "A vademecum on blockchain technologies: When, which, and how," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3796–3838, 4th Quart., 2019.
- [5] D. V. Dimitrov, "Blockchain applications for healthcare data management," *Healthcare Inf. Res.*, vol. 25, no. 1, pp. 51–56, 2019.
- [6] H. Huang, X. Chen, and J. Wang, "Blockchain-based multiple groups data sharing with anonymity and traceability," *Sci. China Inf. Sci.*, vol. 63, no. 3, pp. 1–13, Mar. 2020.
- [7] C. Catalini and J. S. Gans, "Some simple economics of the blockchain," *Commun. ACM*, vol. 63, no. 7, pp. 80–90, Jun. 2020.
- [8] A. Reyna, C. Martín, J. Chen, E. Soler, and M. Díaz, "On blockchain and its integration with IoT. Challenges and opportunities," *Future Gener. Comput. Syst.*, vol. 88, pp. 173–190, Nov. 2018.
- [9] E. Politou, F. Casino, E. Alepis, and C. Patsakis, "Blockchain mutability: Challenges and proposed solutions," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 4, pp. 1972–1986, Oct. 2021.
- [10] Y. Jia, S.-F. Sun, Y. Zhang, Z. Liu, and D. Gu, "Redactable blockchain supporting supervision and self-management," in *Proc. ACM ASIACCS*, May 2021, pp. 844–858.
- [11] G. Ateniese, B. Magri, D. Venturi, and E. Andrade, "Redactable blockchain—Or—Rewriting history in Bitcoin and friends," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroSP)*, Apr. 2017, pp. 111–126.
- [12] G. Ateniese and B. D. Medeiros, "On the key exposure problem in chameleon hashes," in *Proc. SCN*, Berlin, Germany: Springer, 2004, pp. 165–179.
- [13] D. Derler, K. Samelin, D. Slamanig, and C. Striecks, "Fine-grained and controlled rewriting in blockchains: Chameleon-hashing gone attribute-based," in *Proc. NDSS*, 2019, pp. 1–15.
- [14] I. Puddu, A. Dmitrienko, and C. Srdjan, "μchain: How to forget without hard forks," *Cryptol. ePrint Arch.*, Jun. 2020. [Online]. Available: <http://eprint.iacr.org/2017/106>
- [15] X. Li, J. Xu, L. Yin, Y. Lu, Q. Tang, and Z. Zhang, "Escaping from consensus: Instantly redactable blockchain protocols in permissionless setting," *IEEE Trans. Dependable Secure Comput.*, early access, pp. 1–20, Oct. 2022, doi: [10.1109/TDSC.2022.3212601](https://doi.org/10.1109/TDSC.2022.3212601). [Online]. Available: <https://eprint.iacr.org/2021/223>
- [16] D. Deuber, B. Magri, and S. A. K. Thyagarajan, "Redactable blockchain in the permissionless setting," in *Proc. SP*, May 2019, pp. 124–138.
- [17] M. I. Mehar et al., "Understanding a revolutionary and flawed grand experiment in blockchain: The DAO attack," *J. Cases Inf. Technol.*, vol. 21, no. 1, pp. 19–32, 2019.
- [18] M. S. Dousti and A. Küpçü, "Tri-op redactable blockchains with block modification, removal, and insertion," *Turkish J. Electr. Eng. Comput. Sci.*, vol. 30, no. 2, pp. 376–391, Jan. 2022.
- [19] M. S. Dousti and A. Küpçü, "Moderated redactable blockchains: A definitional framework with an efficient construct," in *Proc. DDPM CBT*, Berlin, Germany: Springer, 2020, pp. 355–373.
- [20] D. Grigoriev and V. Shpilrain, "RSA and redactable blockchains," *Int. J. Comput. Math., Comput. Syst. Theory*, vol. 6, no. 1, pp. 1–6, Jan. 2021.
- [21] M. Florian, S. Henningsen, S. Beaucamp, and B. Scheuermann, "Erasing data from blockchain nodes," in *Proc. EuroSPW*, Jun. 2019, pp. 367–376.
- [22] K. Huang, X. Zhang, Y. Mu, F. Rezaeiabgha, and X. Du, "Scalable and redactable blockchain with update and anonymity," *Inf. Sci.*, vol. 546, pp. 25–41, Feb. 2021.
- [23] S. Xu, J. Ning, J. Ma, G. Xu, J. Yuan, and R. H. Deng, "Revocable policy-based chameleon hash," in *Proc. ESORICS*, Berlin, Germany: Springer, 2021, pp. 327–347.
- [24] K. Huang et al., "Building redactable consortium blockchain for industrial Internet-of-Things," *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3670–3679, Jun. 2019.
- [25] J. Xu, K. Xue, H. Tian, J. Hong, D. S. L. Wei, and P. Hong, "An identity management and authentication scheme based on redactable blockchain for mobile networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 6, pp. 6688–6698, Jun. 2020.
- [26] H. Krawczyk and T. Rabin. (1998). *Chameleon Hashing and Signatures*. [Online]. Available: <https://citeseerx.ist.psu.edu/pdf/cd94a5cd939a2f05c892ecaca3713188f4754d63>
- [27] X. Chen, F. Zhang, W. Susilo, H. Tian, J. Li, and K. Kim, "Identity-based chameleon hashing and signatures without key exposure," *Inf. Sci.*, vol. 265, pp. 198–210, May 2014.
- [28] X. Chen et al., "Efficient generic on-line/off-line (threshold) signatures without key exposure," *Inf. Sci.*, vol. 178, no. 21, pp. 4192–4203, Nov. 2008.
- [29] D. Boneh, B. Bünz, and B. Fisch, "Batching techniques for accumulators with applications to IOPs and stateless blockchains," in *Proc. CRYPTO*, Berlin, Germany: Springer, 2019, pp. 561–586.
- [30] J. Camenisch and A. Lysyanskaya, "Dynamic accumulators and application to efficient revocation of anonymous credentials," in *Proc. CRYPTO*, Berlin, Germany: Springer, 2002, pp. 61–76.
- [31] J. Li, N. Li, and R. Xue, "Universal accumulators with efficient non-membership proofs," in *Proc. ACNS*, Berlin, Germany: Springer, 2007, pp. 253–269.
- [32] J. Garay, A. Kiayias, and N. Leonardos, "The Bitcoin backbone protocol: Analysis and applications," in *Proc. EUROCRYPT*, Berlin, Germany: Springer, 2015, pp. 281–310.
- [33] D. Derler, K. Samelin, and D. Slamanig, "Bringing order to chaos: The case of collision-resistant chameleon-hashes," in *Proc. PKC*, Berlin, Germany: Springer, 2020, pp. 462–492.



Jun Shen received the B.S. and M.S. degrees from the Nanjing University of Information Science and Technology, Nanjing, China, in 2015 and 2018, respectively. She is currently pursuing the Ph.D. degree with the School of Cyber Engineering, Xidian University, Xi'an, China. Her research interests include cloud computing security and blockchains.



Xiaofeng Chen (Senior Member, IEEE) received the B.S. and M.S. degrees in mathematics from Northwest University, China, in 1998 and 2000, respectively, and the Ph.D. degree in cryptography from Xidian University in 2003. Currently, he is with Xidian University as a Professor. He has published over 200 research papers in refereed international conferences and journals. His work has been cited more than 10000 times at Google Scholar. His research interests include applied cryptography and cloud computing security. He is an Editorial Board

Member of IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, and *International Journal of Foundations of Computer Science*. He has served as the program/general chair or a program committee member in over 30 international conferences.



Zheli Liu received the B.Sc. and M.Sc. degrees in computer science and the Ph.D. degree in computer application from Jilin University, Changchun, China, in 2002, 2005, and 2009, respectively. After a post-doctoral fellowship with Nankai University, he joined the College of Computer and Control Engineering, Nankai University, in 2011, where he is currently a Professor. His current research interests include applied cryptography and data privacy.



Willy Susilo (Fellow, IEEE) is currently a Distinguished Professor with the Faculty of Engineering and Information Sciences, School of Computing and Information Technology, University of Wollongong (UOW), Australia, where he is also the Director of the School of Computing and Information Technology, Institute of Cybersecurity and Cryptology, and the Head of the School of Computing and Information Technology, UOW. He is a fellow of IET, ACS, and AAIA. He was awarded the prestigious Australian Research Council Future Fellowship in 2009. In 2016, he was awarded the "Researcher of the Year" with UOW. He is also the Editor-in-Chief of the *Computer Standards & Interfaces* (Elsevier) and the *Information* (MDPI). He is also an Associate Editor of IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, *ACM Computing Surveys*, and *Computers & Security* (Elsevier). He has also served as the program committee member for several international conferences.